

Deep Learning Architekturen und Methoden

Contents

Introduction	4
Background.....	4
Computer Vision und Image Classification	4
Historie.....	5
Bildklassifikation	5
Nearest Neighbor	5
Bias/ Variance	6
Linear Classification	6
Loss-Funktion	7
Optimization	7
Numerischer Gradient	8
Gradient Descent.....	8
Konvergenzverhalten	8
Verbesserte Optimierungsverfahren	9
Momentum	9
AdaGrad	9
RMSProp.....	9
Adam (Adaptive Moment Estimation)	10
Backpropagation.....	10
Computational Graph	10
Algorithmus.....	11
Neuronale Netze.....	12
Training Neural Networks	13
Historie	13
Setup	13
Aktivierungsfunktionen	13
Preprocessing	14
Weight Initialisierung	15
Batch Normalisierung.....	15
Trainingsdynamik	16
Babysitting.....	16

Hyperparameter Optimierung	16
Monitoring	16
Evaluation (Ensemble Learning)	17
Methoden	17
Regularisierung (Dropout)	18
Gradient Clipping.....	18
Gradient Noise.....	18
Convolutional Neural Networks	18
Convolution.....	19
Convolution Layers	19
Aufbau	19
Pooling Layer	20
Transfer Learning	20
Fallstudien	21
Bildverarbeitung	22
Objekterkennung	22
R-CNN	23
Fast R-CNN.....	24
Faster R-CNN.....	24
YOLO	25
Evaluation.....	25
Recurrent Networks.....	25
Unrolling.....	25
Training.....	26
Anwendungen.....	26
Long Short-Term Memory (LSTM)	27
Generative Models	28
Pixel RNN/CNN.....	28
Variational Autoencoders (VAEs)	29
Autoencoders	29
Generative Adversarial Networks (GANs)	30
Training.....	30
Deep Probabilistic Models	31
Grundlagen	31
Graphische Modelle.....	32
Bayesian Networks (BNs).....	32

Inferenz.....	32
Markov Networks.....	32
Faktorgraphen.....	32
Sum-Product Networks.....	33
Tractable Probabilistic Models.....	33
Tractable Probabilistic Inference.....	33
Probabilistic Circuits.....	34
Lernen von Probabilistic Circuits.....	35
Anwendungen.....	35
Erweiterungen.....	35
Deep NLP.....	35
Propositional Semantics.....	36
Vektorielle Semantik.....	36
Wort- und Satzrepräsentationen.....	36
Bag-of-Words.....	36
Word Embeddings.....	36
Skip-Thought Vectors.....	36
Siamese Netzwerke.....	37
Maschinelle Übersetzung.....	37
Parsing.....	38
Entity-Relation Extraction.....	38
Evaluation.....	38
Transformers.....	38
Attention.....	39
Architektur.....	39
Anwendungen.....	40
Erweiterungen.....	40

Introduction

Was ist Deep Learning?

- Deep Learning = Teilgebiet des maschinellen Lernens, das **tiefe Architekturen** nutzt.
- Ziel: **Feature-Hierarchien lernen**
 - Niedrige Ebene: Kanten, lokale Formen, Objektteile
 - Höhere Ebene: semantische, abstrakte Repräsentationen
- Meist umgesetzt durch **(gestapelte) neuronale Netze** mit mehreren nichtlinearen Schichten.

Motivation & Beispiele

- **AlphaGo (DeepMind)**: Kombination aus Deep Policy Network + Value Network + Monte Carlo Tree Search.
- Erfolge in:
 - **Bildklassifikation** (AlexNet, 2012, ImageNet)
 - **Spracherkennung** (LSTMs, 1997)
 - **Maschinelle Übersetzung** (Seq2Seq mit Attention)
 - **Spiele** (Atari, Go)
 - **Generative Modelle & Sprachmodelle** (ChatGPT, GPT-4, Aleph Alpha)

Deep Learning: Hype oder Hoffnung?

- **Hype**: Mediale Übertreibungen, große Erwartungen
- **Hoffnung**: Realistische Fortschritte, z. B. Turing Award 2019 (LeCun, Hinton, Bengio)
- Fakt: Deep Learning ist sehr erfolgreich, aber nicht das „menschliche Lernen“.
- **Kritische Fragen**:
 - Warum sind DNNs trotz NP-Härte und Greedy-Optimierung so erfolgreich?
 - Erklärbarkeit: In DNNs haben Knoten/Strukturen keine klar interpretierbare Bedeutung (anders als in probabilistischen Graphischen Modellen).

Historische Entwicklung

- **1940–1960er**: Cybernetics – gehirnähnliche Systeme
- **1960–1980er**: Computer & Komplexitätstheorie → einfache Netzwerke sehr limitiert
- **1980–1990er**: Connectionism, Backpropagation
- **1990–2010er**: Grafische Modelle dominieren, „Winter“ der neuronalen Netze
- **ab 2006**: Wiederaufstieg durch Deep Learning (große Datensätze, GPUs, End-to-End Training)

Background

Computer Vision und Image Classification

- Computer Vision (CV) = automatisierte Extraktion von Information aus Bildern.
- Klassische Ansätze (vor Deep Learning) gingen von handdesignten Features und klar definierten Algorithmen aus.
- Deep Learning bringt hier den Umbruch: Features werden automatisch gelernt.

Historie

- **1970–80er: Marr & Nishihara**
 - Marr: 3 Ebenen der Analyse: *Computation – Algorithm – Implementation*
 - Konzepte wie *Primal Sketch* und *2.5D Sketch* → Repräsentationen von Strukturen im Bild.
 - Binford (1971): *Generalized Cylinders* → frühe Ansätze für Objektrepräsentationen.
 - Marr & Nishihara (1978): Repräsentation durch Skelett- und Zylinderstrukturen.
- **1980er: ACRONYM (Brooks 1982)**
 - Shape Matching durch Zylindermodelle + Suchverfahren.
- **1990er: Eigenfaces (Turk & Pentland 1991)**
 - Erster datengetriebener Ansatz für Gesichtserkennung: Bilder durch PCA repräsentieren.
- **2000er: Invariante Features**
 - Z. B. SIFT (Scale-Invariant Feature Transform).
 - *Bag-of-Words-Modelle* (Sivic & Zisserman, 2003 „Video Google“) → Bilder als Sammlung lokaler Deskriptoren.
- **2010er: Deep Networks**
 - Durch GPUs + große Datensätze wurde *End-to-End Lernen* möglich.
 - Netzwerke lernen Merkmale automatisch, statt sie manuell zu definieren.

Bildklassifikation

Aufgabe: Eingabe = Bild, Ausgabe = Label (z. B. „Katze“, „Hund“, „Auto“).

Problem: Semantische Lücke, Computer sehen Bilder als Arrays (z. B. $300 \times 100 \times 3$, Werte 0–255), Mensch erkennt *Konzepte*, Computer nur *Pixel*

Herausforderungen: Perspektivenwechsel, Beleuchtungsänderung, Deformation, Teilweise Verdeckungen, Hintergrund-Clutter, Intraklassenvariation

Data-driven Ansatz:

1. Datensatz mit Bildern und Labels sammeln
2. Machine Learning benutzen, um einen Bildklassifikator zu trainieren
3. Klassifizierer evaluieren mit einer zurückgehaltenen Testmenge

Nearest Neighbor

Alle Trainingsbilder und die zugehörigen Labels merken und dann das Label welches am ähnlichsten zu einem Trainingsbild ist vorhersagen.

Beispiel-Datensatz: CIFAR-10 (10 Klassen, 50.000 Trainings- und 10.000 Testbilder).

Distanzmaße:

- L1 (Manhattan): $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$
- L2 (Euklidisch): $d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$

Aber: Die Klassifikationsgeschwindigkeit ist linear abhängig von der Trainingsgröße. Gibt auch approximate nearest neighbor, der ist deutlich schneller.

k-Nearest Neighbor

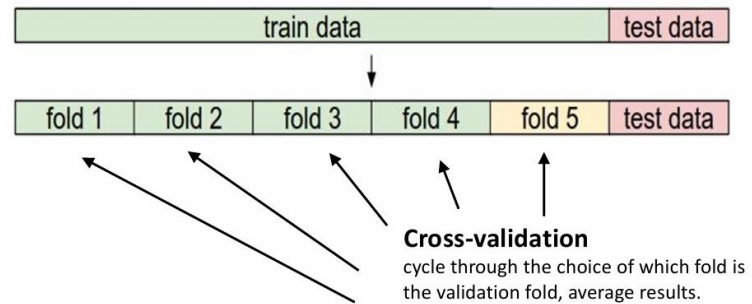
Statt nur den nächsten Nachbarn: Nimm die k nächsten Bilder und lass sie abstimmen. KNN ist robust gegenüber Ausreißern aber die Wahl von k ist entscheidend.

- Größeres $k \rightarrow$ Bias $\uparrow$, Varianz $\downarrow$
- Kleines $k \rightarrow$ Bias $\downarrow$, Varianz $\uparrow$

k -NN Bildverarbeitung:

Schlechtes Verhalten bei Bilddaten, da Distanz in Pixelräumen nicht intuitiv: Bilder, die für Menschen völlig verschieden wirken, können gleiche L2-Distanzen haben. k -NN wird in Computer Vision *nicht* eingesetzt aber wichtig, um

Datengetriebenes Lernen, Bias-Varianz-Tradeoff und Validationset/ Crossvalidation zu verstehen.



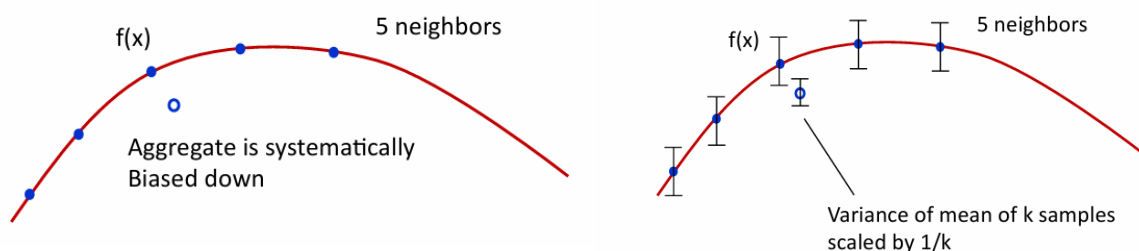
Bias/ Variance

Bias (Verzerrung): systematischer Fehler durch Modellannahmen.

Varianz: Empfindlichkeit gegenüber Trainingsdaten (Rauschen, Ausreißer).

Allgemeine Faustregel:

- Komplexe Modelle $\rightarrow$ niedriger Bias, hoher Varianz (Gefahr von Overfitting).
- Einfache Modelle $\rightarrow$ höherer Bias, niedrige Varianz (Gefahr von Underfitting).
- Deep Networks $\rightarrow$ sehr komplex $\rightarrow$ brauchen große Datenmengen, um Varianz zu kontrollieren.



Linear Classification

Statt das gesamte Trainingsset zu speichern, wird eine **parametrische Funktion** gelernt.

Eingabe x (3072-dim. Vektor für ein Bild) $\rightarrow$ Ausgabe: Score-Vektor für jede Klasse.

Parametrisches Modell: $f(x, W)$ mit Gewichtsmatrix $W \in \mathbb{R}^{K \times D}$ und Bias-Vektor $b \in \mathbb{R}^K$,
 $f(x_i, W, b) = Wx_i + b$

Jedes Gewicht W_{ij} beschreibt die Bedeutung des j -ten Pixels für die i -te Klasse. Es wird die Klasse mit dem höchsten Score gewählt. Allerdings ist die Linearität nicht ausreichend.

Bias-Varianz-Vergleich zu kNN

- **Lineare Klassifikatoren:**
 - Höherer **Bias** (Fehler bei nicht-linear separierbaren Daten).
 - Geringere **Varianz** (stabil gegenüber Rauschen, da Gewichtung über alle Pixel).

- **k-NN:**
 - Geringerer Bias (nahezu beliebig komplex).
 - Höhere Varianz (stark abhängig von Trainingsdaten).

Loss-Funktion

Parameter W, b_W, b so wählen, dass die Klassifikationsfehler minimal sind. Dazu wird eine **Loss-Funktion** definiert.

Multiclass SVM Loss

Nehmen wir ein Beispiel: (x_i, y_i) wobei x_i das Bild und y_i die (ganzzahlige) Beschriftung ist, und verwenden wir die Kurzform für den Punktestandvektor: $s = f(x_i, W)$ der SVM Loss hat die Form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Und der gesamte Trainingsloss ist der Durchschnitt über alle Beispiele: $L = \frac{1}{N} \sum_{i=1}^N L_i$

Weight Regularization

Weight Regularization wird benutzt um zu vermeiden, dass es für $L=0$ mehrere W gibt.

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i, W)_j - f(x_i, W)_{y_i} + 1) + \lambda R(W)$$

- L2 Regulierung: $R(W) = \sum_k \sum_l W_{k,l}^2$
- L1 Regulierung: $R(W) = \sum_k \sum_l |W_{k,l}|$
- Elastic Net (L1+L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$
- Dropout
- Max Norm Regulierung

Softmax Classifier

$P(Y = k | X = x_i) = \frac{e^{s_{y_i}}}{\sum_j e^{s_j}}$, wobei $s = f(x, W)$ die Scores sind (nicht-normalisierte logarithmische Wahrscheinlichkeit der Klassen)

Wir wollen die Log Likelihood maximieren oder (für eine Lossfunktion) die negative log Likelihood der richtige Klasse minimieren: $L_i = -\log P(Y = y_i | X = x_i) = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$

SVM definiert die Margin zwischen Klassen und ist robust gegenüber kleinen Änderungen der Scores. Der Softmax gibt Wahrscheinlichkeitsaussagen und ist sensitiver gegenüber Score-Änderungen. Die Wahl ist abhängig von der Anwendung: Klassifikation vs probabilistische Modellierung.

Optimization

Naiver Ansatz: zufällige Gewichtsmatrizen ausprobieren, führt nur zu 15% Genauigkeit.

Numerischer Gradient

Ableitung der Loss-Funktion nutzen, der Gradient ist der Vektor der partiellen Ableitungen und gibt die Richtung des stärksten Anstiegs an. Die Optimierung erfolgt durch den Schritt in negative Gradientenrichtung.

Der numerische Gradient ist die Approximation der Ableitung durch kleine Differenzen:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Der numerische Gradient ist einfach zu implementieren und robust aber extrem langsam und ungenau für große Modelle.

Der Analytische Gradient ist exakt und effizient aber fehleranfällig, daher macht man in der Praxis Gradient Checking (numerische Approximation zur Validierung des analytischen Gradienten). Der analytische Gradient wird mit Differenzialrechnung hergeleitet (z.B. über die Kettenregel).

Gradient Descent

Der Gradientenabstieg ist ein iterativer Optimierungsalgorithmus, der eine Funktion minimiert, indem er Schritte in die entgegengesetzte Richtung des Gradienten der Funktion (der „steilste Abstieg“) unternimmt.

Batch Gradient Descent

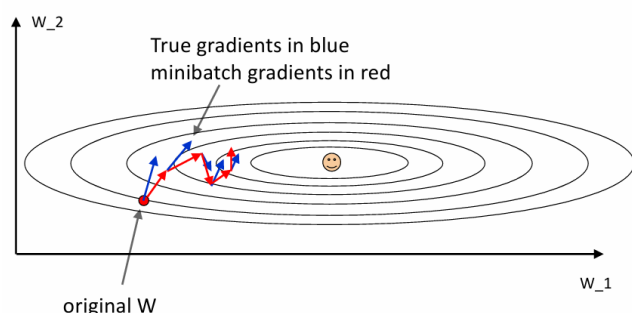
- Berechnung des Gradienten über **gesamtes Trainingsset**.
- Genau, aber teuer bei großen Datensätzen.

Mini-Batch Gradient Descent

- Gradient wird über kleine Teilmengen berechnet (z. B. 32, 64, 128 Beispiele).
- Vorteile: schneller, nutzt Vektoroperationen, stabilisiert Rauschen.
- Standard in Deep Learning.

Stochastic Gradient Descent (SGD)

- Extrem kleiner Batch (oft 1 Beispiel).
- Starke Varianz im Gradienten, aber guter Fortschritt im Mittel.
- Günstig in der Berechnung.



Konvergenzverhalten

- **Newton-Verfahren:** Quadratische Konvergenz ($O(\log \log n)$), sehr schnell, aber unpraktisch (Hessian $O(M^3)$ zu teuer, numerisch instabil).
- **Gradient Descent:** Lineare Konvergenz ($O(\log n)$).
- **SGD:** Sublineare Konvergenz $O(1/n)$, aber ausreichend für ML, da nach Millionen Updates Genauigkeit im Bereich der Maschinenpräzision liegt.

Approximation L-BFGS: Näherungsverfahren für Newton, speichert begrenzte Information über die Krümmung. Weniger Speicher- und Rechenaufwand.

Verbesserte Optimierungsverfahren

Momentum

Akkumulation eines Geschwindigkeitsvektors in Gradientenrichtung. Führt zu schneller Konvergenz in flachen Richtungen.

$$v_t = \mu v_{t-1} - \eta \nabla L(W_t), W_{t+1} = W_t + v_t$$

Wobei $\mu \in [0.5, 0.99]$

```
# Momentum update
v = mu * v - learning_rate * dx # integrate velocity
x += v # integrate position
```

Nestrov Momentum

„Lookahead“-Variante: Der Gradient wird am vorhergesagten nächsten Punkt berechnet. Führt zu präziseren Updates und besserer Konvergenz als Standard-Momentum.

$$v_t = \mu v_{t-1} - \epsilon \nabla f(\theta_{t-1} + \mu v_{t-1}), \theta_t = \theta_{t-1} + v_t$$

```
# Nesterov momentum update rewrite
v_prev = v
v = mu * v - learning_rate * dx
x += -mu * v_prev + (1 + mu) * v
```

AdaGrad

Anpassung der Lernrate pro Dimension basierend auf historischen Gradienten:

$$W_{t+1} = W_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \nabla L(W_t)$$

Mit $G_t = \sum_{\tau=1}^t (\nabla L(W_\tau))^2$

Vorteil: seltene Features erhalten größere Updates.

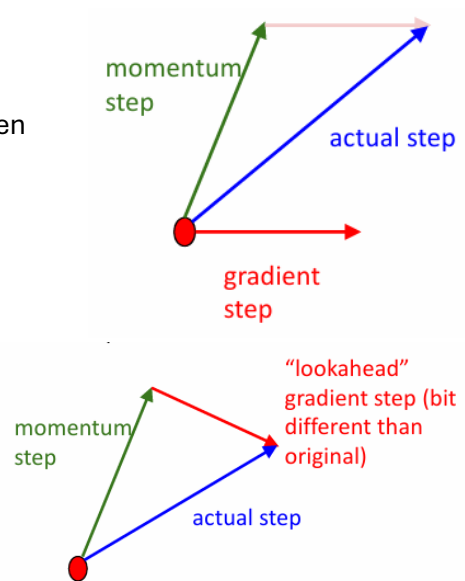
Nachteil: Lernrate schrumpft im Laufe des Trainings stark.

```
# Adagrad update
cache += dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```

RMSProp

Modifikation von AdaGrad: Verwendet exponentiell gleitenden Durchschnitt statt kumulativer Summe und verhindert zu starkes Abfallen der Lernrate.

```
# RMSProp
cache = decay_rate * cache + (1 - decay_rate) * dx**2
x += - learning_rate * dx / (np.sqrt(cache) + 1e-7)
```



Adam (Adaptive Moment Estimation)

Kombination aus Momentum und RMSProp

```
# Adam
m,v = #... initialize caches to zeros
for t in xrange(1, big number):
    dx = # ... evaluate gradient
    m = beta1*m + (1-beta1)*dx # update first moment
    v = beta2*v + (1-beta2)*(dx**2) # update second moment
    mb = m/(1-beta1**t) # correct bias
    vb = v/(1-beta2**t) # correct bias
    x += - learning rate * mb / (np.sqrt(vb) + 1e-7)
```

momentum

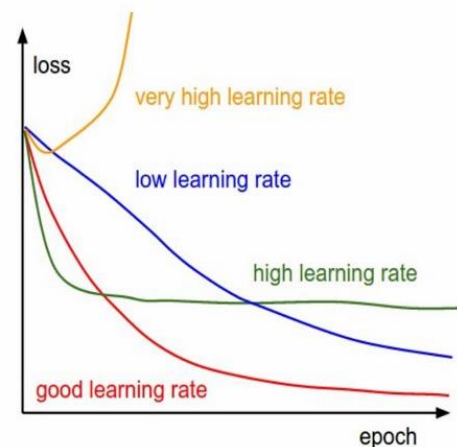
bias correction
(only relevant in first few iterations when t is small)

RMSProp-like

Die Wahl der Lernrate η oder α ist entscheidend. Zu große Lernrate führt zu Divergenz oder Oszillationen und zu kleine Lernrate führt zu langsamer Konvergenz.

Lernratenscheduling:

- Step Decay: periodische Halbierung alle paar Epochen.
- Exponentieller Abfall: $\alpha = \alpha_0 e^{-kt}$
- 1/t-decay: $\alpha = \frac{\alpha_0}{1+kt}$



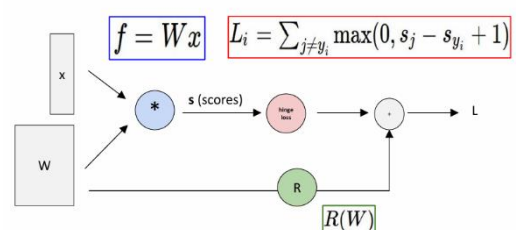
Backpropagation

Backpropagation ist ein Algorithmus, der auf der **Kettenregel der Differenzialrechnung** basiert und eine effiziente Gradientenberechnung in **computational graphs** ermöglicht. Das Ziel ist die Berechnung der Gradienten von **Scores** und **Loss-Funktion** bezüglich aller Modellparameter und Nutzung dieser Gradienten in **SGD und Varianten** zur Optimierung.

Computational Graph

Ein **computational graph** ist eine strukturierte Darstellung einer Funktion, bestehend aus:

- Knoten (Operationen oder Variablen),
- gerichteten Kanten (Datenfluss zwischen Operationen).



Beispiel:

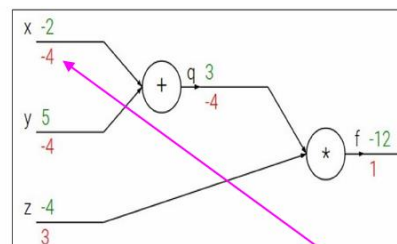
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

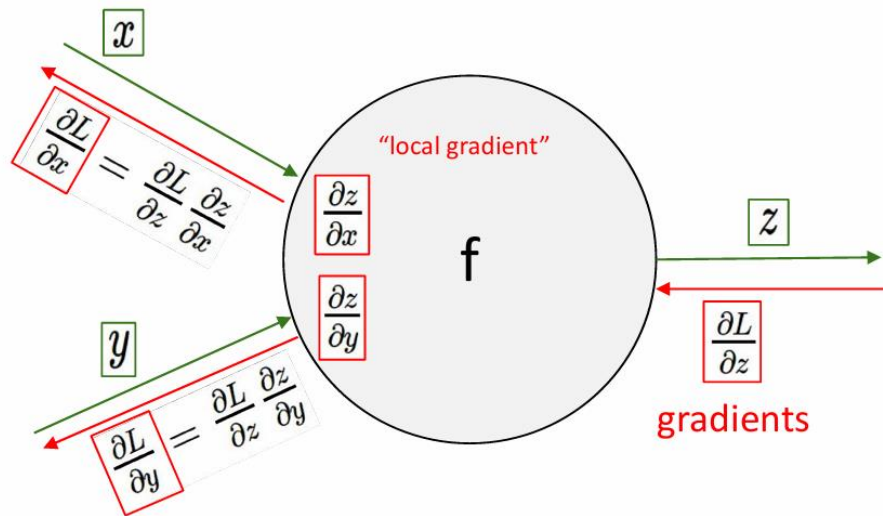


Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

$$\frac{\partial f}{\partial x}$$

Die Grundlage der Backpropagation ist die Kettenregel. Wenn eine Funktion in Teilschritte zerlegt ist (z. B. $f(x, y, z) = g(h(x, y), z)$) lässt sich die Ableitung schrittweise bestimmen.

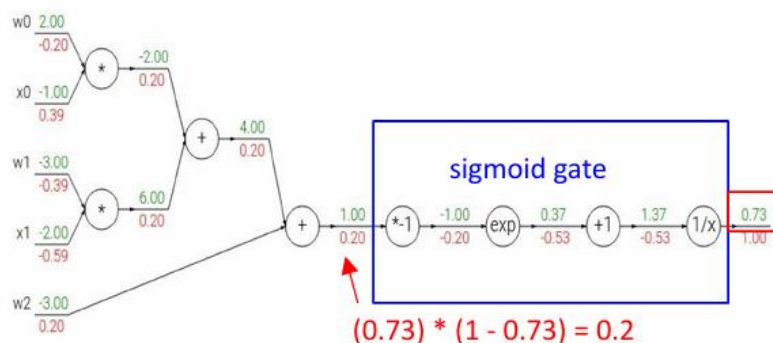


Jede elementare Operation im Graph besitzt eine einfache Ableitungsregel:

1. **Addition:** verteilt den Gradienten unverändert auf beide Eingänge.
2. **Multiplikation:** Gradient am Eingang = Produkt des Gradientenflusses mit dem jeweils anderen Faktor.
3. **Max-Gate:** leitet den Gradienten nur an den „gewonnenen“ Eingang weiter.
4. **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$, $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2 x_2)}} \quad \sigma(x) = \frac{1}{1 + e^{-x}} \quad \text{sigmoid function}$$

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$



Algorithmus

1. **Vorwärtsthroughlauf (forward pass):**
 - Berechne alle Zwischenergebnisse und den Loss.
 - Speichere Werte, die für die Gradientenberechnung benötigt werden.
2. **Rückwärtsthroughlauf (backward pass):**

- Starte mit $\frac{\partial L}{\partial L} = 1$.
- Wende die Kettenregel rekursiv rückwärts im Graphen an.
- Berechne Gradienten für alle Knoten und Parameter.

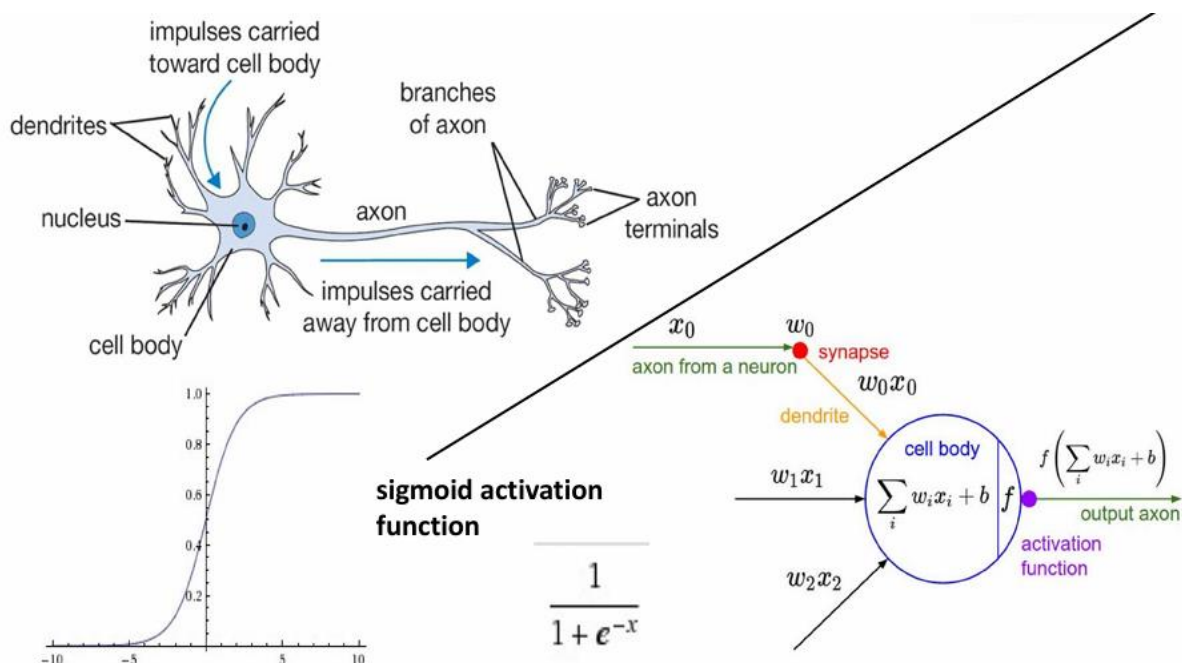
Backpropagation ist reverse-mode differentiation. Sie ist Vorteilhaft, wenn viele Eingaben, aber nur eine skalare Ausgabe (Loss) benötigt wird. Im Gegensatz dazu: **forward-mode differentiation** für Szenarien mit wenigen Eingaben und vielen Ausgaben.

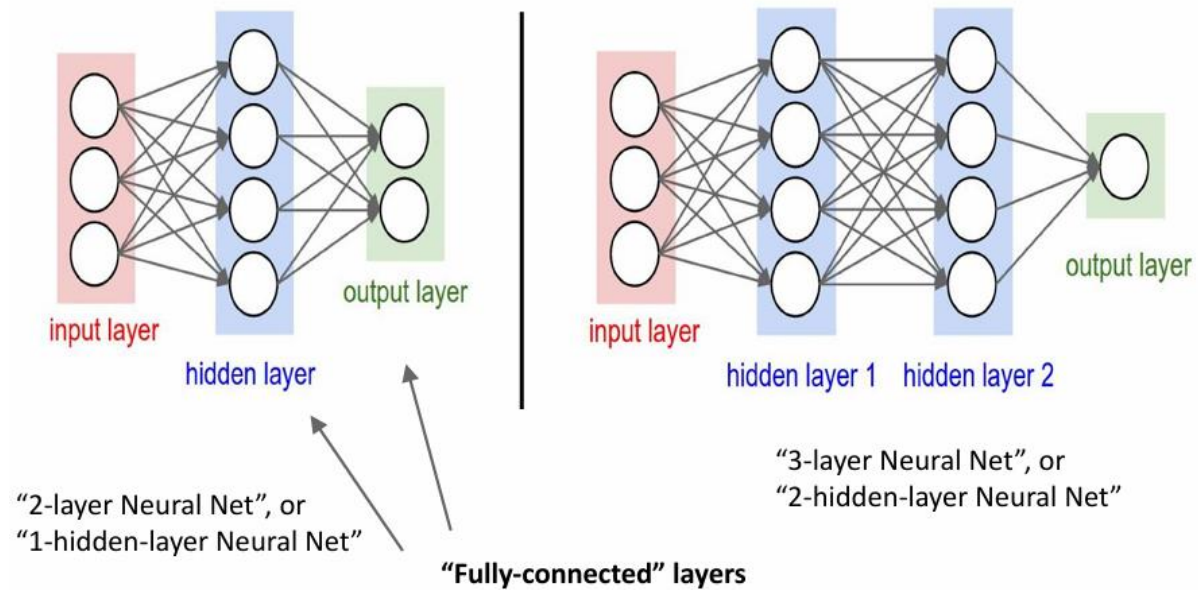
Neuronale Netze

Bei neuronalen Netzen arbeiten wir mit Vektoren und Matrizen. Die Aktivierungsfunktionen werden **elementweise** angewendet (z. B. ReLU). In der Praxis wird die volle Matrix nicht berechnet, sondern Vektorisierung genutzt → effiziente Matrixoperationen. Die Formal ergibt sich die Ableitung als **Jacobian-Matrix**. Neuronale Netze sind hochdimensional; manuelles Aufschreiben aller Gradienten ist unmöglich. Frameworks implementieren forward-/backward-APIs zur Automatisierung. Vollständig verbundene Layer ermöglichen effiziente Darstellung durch Matrixmultiplikationen. Mehr Neuronen = mehr Modellkapazität → Gefahr von Overfitting, erfordert Regularisierung.

Aufbau:

- **Lineare Klassifikatoren:** einfache Gewichtsmatrix $f = Wx$
- **2-Layer-Netzwerk:** Input → Hidden Layer (mit Aktivierung) → Output Layer. $f = W_2 \max(0, W_1 x)$
- **Mehrschichtige Netzwerke (MLPs):** Wiederholung dieser Struktur. 3-Layer: $f = W_3 \max(0, W_2 \max(0, W_1 x))$





Training Neural Networks

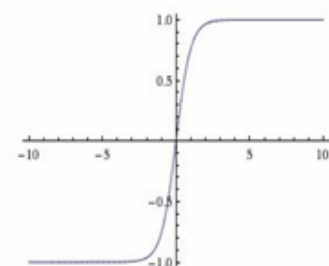
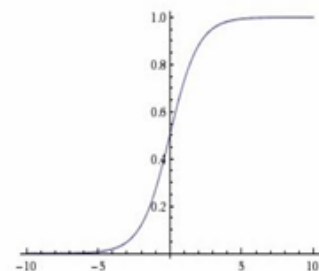
Historie

- **1957, Rosenblatt:** Perceptron – erstes trainierbares Modell, basierend auf linearen Entscheidungsgrenzen.
- **1960, Widrow & Hoff:** Adaline/Madaline – Erweiterung mit adaptiven linearen Neuronen.
- **1986, Rumelhart et al.:** Backpropagation erstmals populär.
- **2006, Hinton & Salakhutdinov:** Neue Begeisterung durch Pretraining tiefer Netze.
- **2010, Dahl et al.:** Erste starke Ergebnisse im Bereich Sprachverarbeitung.
- **2012, Krizhevsky et al.:** AlexNet revolutioniert Bildklassifikation mit CNNs.

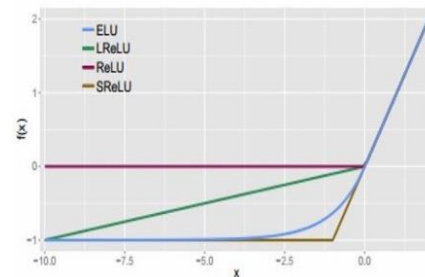
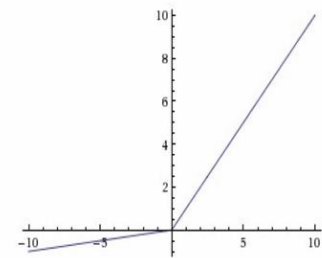
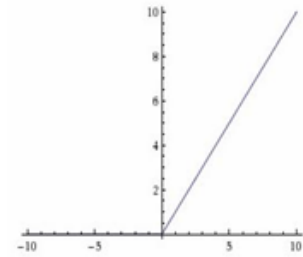
Setup

Aktivierungsfunktionen

- **Sigmoid:** $\sigma(x) = \frac{1}{1+e^{-x}}$, $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
 - Komprimiert Zahlen auf den Bereich $[0, 1]$ – kann Gradienten zerstören.
 - Ein Schlüsselement in LSTM-Netzwerken „Steuersignale“
 - Am besten geeignet für das Lernen „logischer“ Funktionen, d. h. Funktionen mit binären Eingaben.
 - Nicht so gut geeignet für Bildnetzwerke (ersetzt durch RELU)
 - Nicht nullzentriert
- **tanh:** ähnlich wie Sigmoid, aber Wertebereich $[-1, 1]$.
 - Komprimiert Zahlen auf den Bereich $[-1, 1]$
 - Nullzentriert (gut)



- Unterdrückt jedoch weiterhin Gradienten bei Sättigung :(
- Wird auch in LSTMs für begrenzte, vorzeichenbehaftete Werte verwendet.
- Nicht so gut für binäre Funktionen geeignet
- **ReLU:** $f(x) = \max(0, x)$, Gradient = 1 für $x > 0$, sonst 0
 - Sättigt nicht (in +Region)
 - Konvergiert schneller als Sigmoid/Tanh bei Bilddaten (z. B. $\sim 6x$)
 - Nicht für logische Funktionen geeignet
 - Nicht für die Steuerung in rekurrenten Netzen geeignet
 - Nicht nullzentrierte Ausgabe
- **Leaky Relu:** $f(x) = \max(0.1x, x)$,
 - Sättigt nicht
 - Konvergiert schneller als Sigmoid/Tanh bei Bilddaten (z. B. $6x$)
 - „stirbt“ nicht
- **Parametric Rectifier (PReLU):** $f(x) = \max(\alpha x, x)$
- **Exponential Linear Units (ELU):** $f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$
 - Alle Vorteile von ReLU
 - Stirbt nicht
 - Näher am Nullmittelwert
- **Maxout:** $f(x) = \max(w_1^T x + b_1, w_2^T x + b_2)$
 - Hat nicht die Grundform des Skalarprodukts -> Nichtlinearität.
 - Verallgemeinert ReLU und Leaky ReLU.
 - Lineares Verhalten! Sättigt nicht! Stirbt nicht ab!
 - Problem: Doppelte Anzahl von Parametern/ Neuronen



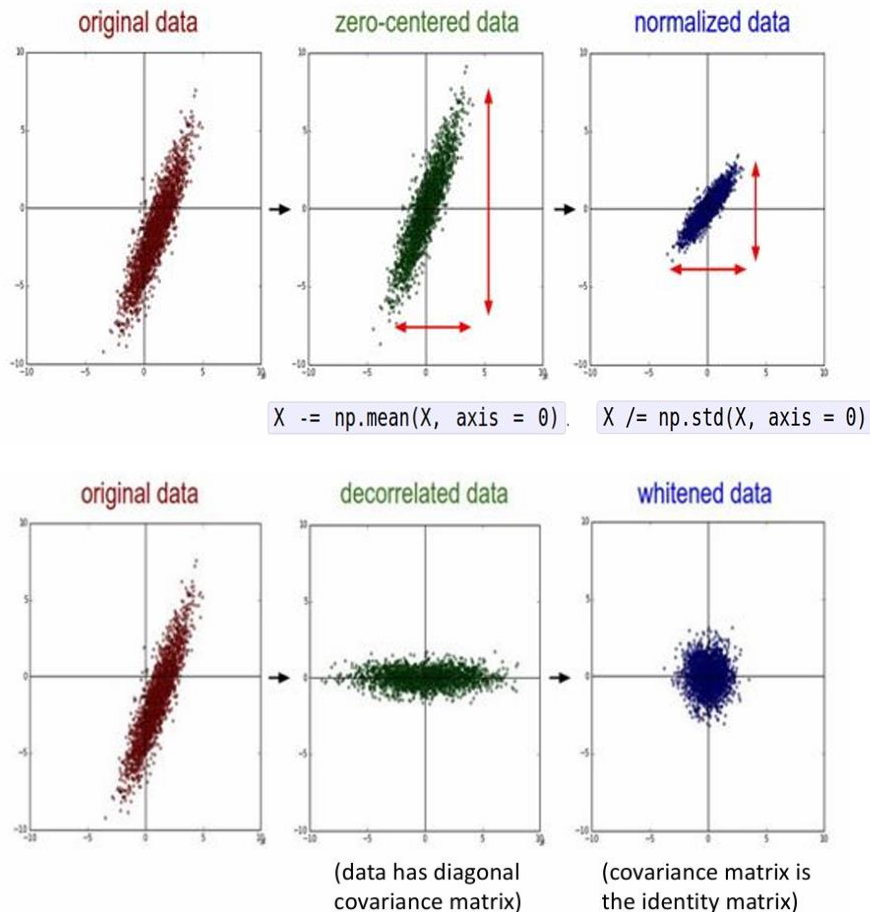
In Praxis, alles versuchen, typischerweise:

- ReLU für Bilddaten.
- Leaky ReLU / ELU / Maxout ausprobieren.
- Sigmoid für logische Aufgaben.
- tanh für signed values.

Preprocessing

Für stabile und effiziente Optimierung müssen Eingabedaten normiert werden:

- **Mean-Centering:**
 - Abzug des Mittelwerts der Eingabedaten.
 - Varianten: globales Mean-Image (z. B. AlexNet) oder kanalweise Mittelwertabzüge (z. B. VGG).
- **Varianznormalisierung, PCA, Whitening:**
 - In Praxis bei Bildern weniger verbreitet.
 - Meist reicht Zentrierung.



Weight Initialisierung

- **Falsche Initialisierung (z. B. nur Nullen):** führt zu Symmetrieproblemen.
- **Kleine Zufallswerte:** funktionieren bei kleinen Netzen, führen aber bei tiefen Netzen zu saturierten Aktivierungen und verschwindenden Gradienten.
- **Xavier-Initialisierung (Glorot et al. 2010):**
 - Spezielle Verteilung, die Aktivierungsvarianz konstant hält.
 - Gut bei tanh, weniger bei ReLU.
- **He-Initialisierung (2015):**
 - Anpassung für ReLU-Netze, skaliert stärker.
 - Ermöglicht stabiles Training tiefer Netze.

Initialisierung ist weiterhin ein aktives Forschungsfeld, jedoch gelten Xavier und He als Standard.

Batch Normalisierung

- Normiert die Aktivierungen innerhalb jedes Batches.
- Schritte:
 1. Berechne Mittelwert und Varianz pro Dimension.
 2. Normalisiere auf Mittelwert 0 und Varianz 1.
 3. Skalierung und Verschiebung durch lernbare Parameter erlaubt.
- Vorteile:
 - Stabilerer Gradientenfluss.
 - Höhere Lernraten möglich.
 - Weniger sensitiv gegenüber Initialisierung.

- Wirkt teilweise wie Regularisierung.
- Testzeit: BN verwendet gespeicherte Mittelwerte/Varianzen aus dem Training (running averages).

Trainingsdynamik

Babysitting

- **Sanity Checks:** Loss ohne Regularisierung ≈ 2.3 für 10 Klassen (Softmax).
- **Regulierungstest:** Erhöht man die Regularisierung, steigt der Loss $\rightarrow$ korrekte Implementierung.
- **Overfitting-Test:** Netz sollte kleine Trainingsmengen überfitten können (Loss $\rightarrow 0$, Accuracy $\rightarrow 100\%$).

Hyperparameter Optimierung

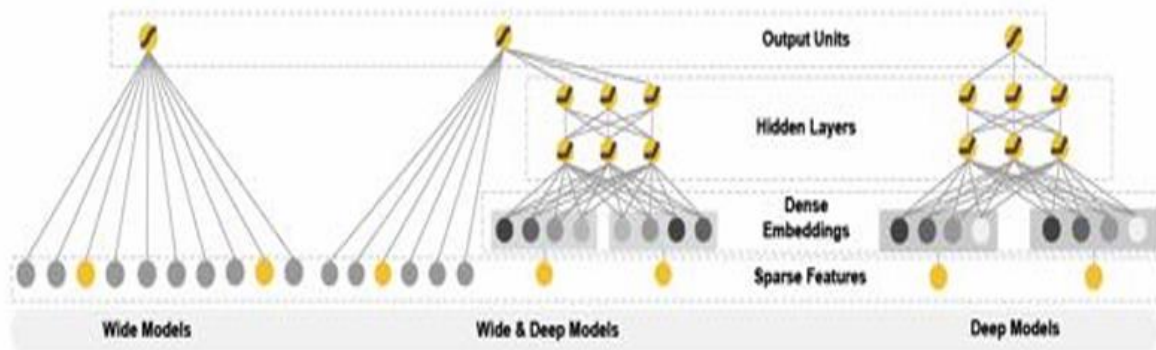
- **Lernrate:** zentraler Parameter.
 - Zu niedrig $\rightarrow$ langsame Konvergenz.
 - Zu hoch $\rightarrow$ Divergenz, NaN-Werte.
 - Typische Suche: $10^{-3} \dots 10^{-5}$.
- **Cross-Validation:**
 - Grobe Suche mit wenigen Epochen.
 - Feinsuche im relevanten Bereich.
 - Praktisch: Hyperparameter in **log-Space** durchsuchen.
- **Random Search vs. Grid Search:**
 - Random Search oft effizienter.
- Neuronale Netze besitzen zahlreiche Hyperparameter:
 - Architektur (Anzahl Layer, Anzahl Neuronen pro Layer),
 - Lernrate und deren Abfall (schedule),
 - Optimierungsverfahren (SGD, Adam, RMSProp etc.),
 - Regularisierungsstärken (L2, Dropout-Rate).
- Effektive Strategien:
 - **Coarse-to-Fine Search:** Zunächst grobe Rasterung mit wenigen Epochen, dann Feinabstimmung im relevanten Bereich.
 - **Random Search statt Grid Search:** In hoher Dimensionalität deutlich effizienter, da Grid Search Ressourcen auf unwichtige Dimensionen verschwendet.
 - **Log-Space Suche:** Hyperparameter wie Lernrate variieren über Größenordnungen. Suche in linearer Skala ineffektiv.

Monitoring

- **Loss-Kurven:** sollten stabil abfallen; starke Anstiege = Lernrate zu hoch.
- **Accuracy:**
 - Große Lücke zwischen Training und Validierung $\rightarrow$ Overfitting.
 - Keine Lücke, niedrige Accuracy $\rightarrow$ zu geringe Modellkapazität.
- **Update-to-Weight-Ratio:** Verhältnis von Update-Größe zu Gewichtswert sollte ca. 0.001 sein.

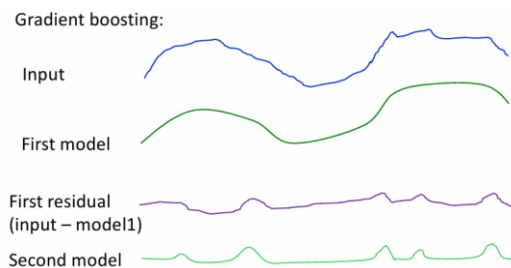
Evaluation (Ensemble Learning)

Ein Ensemble ist ein Modell, das durch die Kombination mehrerer schwächerer Modelle entsteht. Ziel ist es, die Generalisierung zu verbessern. Prognose durch Mittelung (Regression) oder Mehrheitsvotum (Klassifikation). Wichtiger Effekt: Verringerung der Varianz der Vorhersagen.



Methoden

- **Bagging (Bootstrap Aggregation):**
 - Jedes Basismodell wird auf einer Bootstrap-Stichprobe trainiert.
 - Endergebnis = Mehrheitsvotum oder Mittelwert.
 - Klassisches Beispiel: Random Forests.
- **Boosting:**
 - Basismodelle werden sequenziell trainiert.
 - Jedes nachfolgende Modell konzentriert sich auf Fehler der Vorgänger.
 - Beispiele: AdaBoost (Gewichtung fehlerhafter Beispiele), Gradient Boosting (Training auf Residuen).
 - Reduziert Bias und teilweise auch Varianz.
- **Stacking:**
 - Kombiniert Basismodelle mit unterschiedlichen Gewichtungen, oft über ein zusätzliches Metamodell.



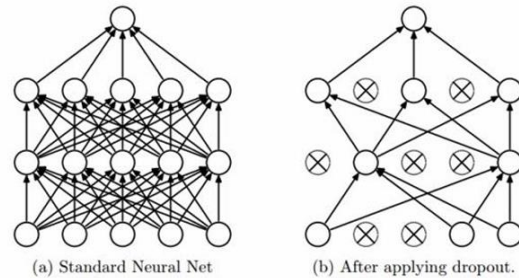
Anwendung: Bagging kann Accuracy bei Bildklassifikation um wenige Prozent steigern. Boosting wird im Deep Learning selten eingesetzt, da neuronale Netze schwer sequentiell „korrigierbar“ sind. Neuere Ansätze: Kombination von **Wide & Deep Models** – breite Modelle für lokale Effekte, tiefe Modelle für globale Strukturen.

- Mehrere **Checkpoints eines Modells** mitteln → kleine Performancesteigerung.
- Verwendung von **Laufzeitdurchschnitten der Gewichte** kann Stabilität und Genauigkeit erhöhen.

Regularisierung (Dropout)

Dropout: In jeder Forward-Pass-Iteration wird zufällig ein Teil der Neuronen auf 0 gesetzt.

- Dies zwingt das Netzwerk, redundante Repräsentationen zu lernen.
- Jede zufällige Maskierung entspricht einem eigenen Modell $\rightarrow$ Dropout $\approx$ Ensemble aus exponentiell vielen Netzen mit Parametererteilung.
- **Training:** zufällige Masken, Neuronen fallen mit Wahrscheinlichkeit p aus.
 - Während Training: Erwartungswert der Aktivierung reduziert um Faktor p .
- **Test:** keine Neuronen deaktiviert $\rightarrow$ stattdessen Skalierung der Aktivierungen, um Erwartungswerte auszugleichen.
 - Während Test: Aktivierungen um Faktor $1 - p$ skaliert, sodass Erwartung übereinstimmt.



Gradient Clipping

Zu große Gradienten führen zu Divergenz des Trainings und zu kleine Gradienten führen zu langsamer Konvergenz. Was besonders kritisch in tiefen Netzen und RNNs

Beim Gradient Clipping werden die Gradienten auf eine maximale Norm begrenzt: $\bar{g}_i = \min(g_{max}, \max(-g_{max}, g_i))$ und verhindert das Explodieren der Gradienten und stabilisiert das Training.

Extremere Variante: Beim One-Bit Gradient werden alle Gradienten auf ± 1 (nur Vorzeichen bleibt erhalten) skaliert. Überraschenderweise funktioniert dies in bestimmten Szenarien (z. B. Sprachmodelle) mit geringer Genauigkeitseinbuße. Anwendung in **verteiletem Training** (Bandbreitenreduktion).

Gradient Noise

SGD produziert von Natur aus verrauschte Gradienten, was hilfreich sein kann, um lokale Minima zu verlassen. Deswegen gezielt zusätzliches Rauschen hinzufügen.

Hinzufügen von Gaußischem Rauschen zum Gradienten: $g' = g + \mathcal{N}(0, \sigma^2(t))$ Die Varianz $\sigma^2(t)$ wird über das Training gesteuert (z.B. abnehmend).

Diese Methode fördert Exploration des Parameterraums und hilft flache Minima zu finden (bessere Generalisierung). Gradient Noise ist besonders nützlich bei sehr tiefen Netzen, bei denen Overfitting droht.

Gradient Noise kann als bayesianische Approximation gesehen werden: Rauschgröße kontrolliert „Temperatur“ der Posterior-Verteilung und das Momentum wirkt ergänzend, indem es den Detailgrad der Suche steuert.

Convolutional Neural Networks

Convolutional Neural Networks (CNNs, auch ConvNets genannt) sind spezialisierte neuronale Netze für die Verarbeitung von visuellen Daten. Sie basieren auf drei zentralen Ideen:

1. **Lokale Rezeptive Felder:** Neuronen reagieren nur auf lokale Regionen des Eingaberaums.
2. **Gewichtsteilung (Parameter Sharing):** dieselben Filter (Kerne) werden über die gesamte Eingabe angewendet.
3. **Hierarchische Merkmalsrepräsentationen:** frühe Schichten detektieren einfache Strukturen (Kanten, Texturen), tiefere Schichten komplexe Objekte.

CNNs stellen damit eine skalierbare und robuste Architektur für Bild- und Videodaten dar.

Convolution

Eine **Faltung** ist die Anwendung eines Filters (Kerns) auf lokale Regionen der Eingabe.

$$(f * g)(i, j) = \sum_m \sum_n f(m, n) \cdot g(i - m, j - n)$$

Praktisch: in CNNs „slidet“ ein kleiner Filter (z. B. 3×3 , 5×5) über das Bild.

Ergebnis: **Feature Map** oder **Aktivierungskarte**, die anzeigt, wo ein bestimmtes Muster vorkommt.

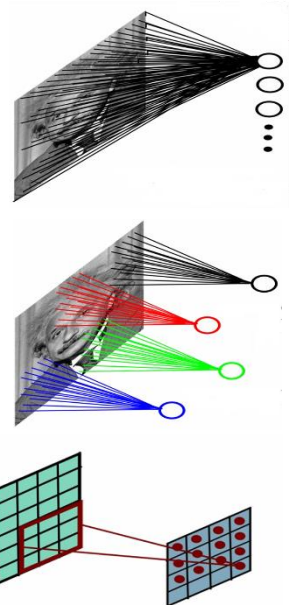
Diese Filter können z.B. eine Glättung (Blur), Kantendetektion (Sobel-Filter) oder Schärfung sein.

Convolution Layers

Fully Connected Layer: Jeder Neuron ist mit allen Pixeln verbunden aber dadurch hohe Anzahl an Parametern. (200x200 Bild, 40.000 Eingaben und Neuronen = 2 Milliarden Parameter)

Locally Connected Layer: Verbindungen nur innerhalb lokaler Regionen, bei 10x10 Filtergröße nur ca 4 Mio Parameter. Aber keine Gewichtsteilung und die Parameteranzahl hängt stark von der Bildgröße ab.

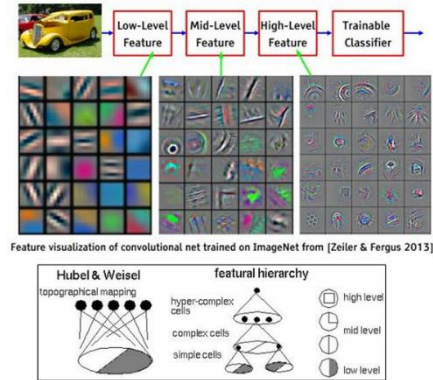
Convolutional Layer: Der selbe Filter wird über alle Positionen angewendet (Parameter Sharing). Das reduziert die Parameteranzahl enorm und nutzt die Translationsinvarianz (Merkmale unabhängig von Position) und bietet somit bessere Generalisierung.



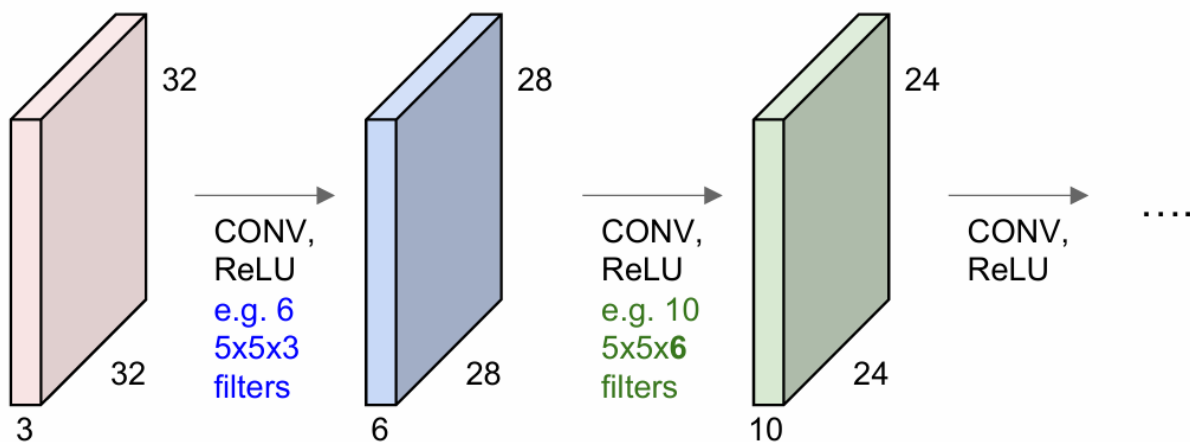
Aufbau

- Eingabevolumen: $W_1 \times H_1 \times D_1$
- Hyperparameter:
 - Anzahl Filter K
 - Filtergröße F
 - Stride S
 - Null-Padding P
- Produziert ein Ausgangsvolumen von $W_2 \times H_2 \times D_2$ mit
 - $W_2 = \frac{W_1 - F + 2P}{S} + 1$
 - $H_2 = \frac{H_1 - F + 2P}{S} + 1$
 - $D_2 = K$

- Filter: $F \times F \times D$, wichtig ist, dass der Filter die gleiche Tiefe hat wie das Eingabevolumen
- Mit Parameter Sharing: Weights pro Filter: $F \cdot F \cdot D_1$ und insgesamt $(F \cdot F \cdot D_1) \cdot K$ Weights und K Biases
- Parameteranzahl: $(F \cdot F \cdot D_1 + 1) \cdot K$ (+1 für Bias)
- Im Ausgabevolumen ist die d-te Schicht (mit Größe $W_2 \times H_2$) das Ergebnis einer validen Faltung des d-ten Filters mit einem Stride S und dann mit Offset d-ten Bias.



Beispiel: Eingabegröße $32 \times 32 \times 3$ und Filter $5 \times 5 \times 3$ führt zu einer Activation Map mit der Größe $28 \times 28 \times 1$, wenn wir nun 6 verschiedenen Filter verwenden, erhalten wir am Ende Eine Ausgabe der Größe $28 \times 28 \times 6$



ConvNet ist eine Sequenz von Convolutional Layern mit Activation Functions dazwischen.

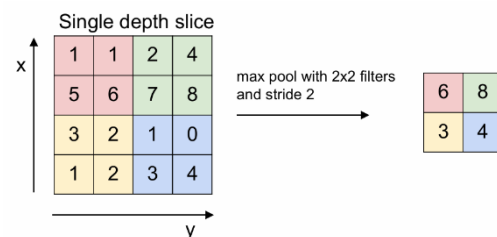
Typischer Aufbau eines CNN: $((\text{CONV-RELU}) \cdot N \cdot \text{POOL?}) \cdot M \cdot (\text{FC-RELU}) \cdot K, \text{SOFTMAX}$, mit N bis zu 5, M ist groß und K zwischen 0 und 2

Pooling Layer

Pooling reduziert die räumliche Auflösung und erhöht Robustheit:

- **Max-Pooling:** größter Wert im Fenster.
- **Average-Pooling:** Mittelwert im Fenster.

Beispiel: 2×2 Max-Pooling mit Stride 2 $\rightarrow$ Halbierung der Höhe und Breite.



Der Pooling Layer reduziert die Parameter und Rechenlast und erhöht die Invarianz gegenüber Verschiebungen.

Transfer Learning

Training großer CNNs erfordert Millionen Bilder.

Transfer Learning: Vorgehen bei kleineren Datensätzen.

1. Pretraining auf großen Datensätzen (z. B. ImageNet).
2. Einsatz des Netzes als Feature-Extraktor.

3. Fine-Tuning einzelner Schichten bei genügend Daten.

CNN-Features lassen sich für viele Aufgaben wiederverwenden (Generische Repräsentationen).

- **Small dataset:** CNN als Feature-Extraktor verwenden, nur den Klassifikator neu trainieren (Softmax-Layer ersetzen).
- **Medium dataset:** Fine-Tuning – alte Gewichte als Initialisierung, Training aller oder nur höherer Schichten. Lernrate: ca. 1/10 für obere Layer, 1/100 für tiefe Layer.
- **Large dataset:** komplettes Training von Grund auf.

Fallstudien

LeNet-5 (LeCun et al., 1998)

- Anwendung: Handschriftenerkennung (MNIST).
- Architektur: CONV → POOL → CONV → POOL → FC → FC → Softmax.
- Filtergröße: 5×5, Pooling: 2×2.
- Wegbereiter für moderne CNNs.

AlexNet (Krizhevsky et al., 2012)

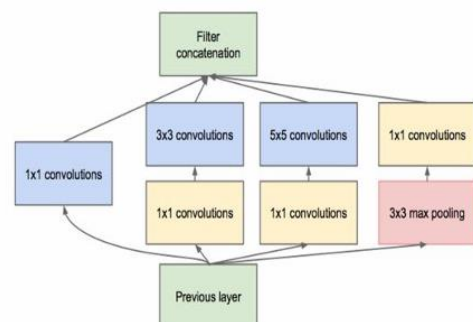
- Revolutionärer Erfolg bei ILSVRC 2012.
- Input: 227×227×3.
- Erste Schicht: 96 Filter 11×11, Stride 4.
- Architektur: abwechselnd CONV, POOL, Normalisierung, FC.
- Key Features:
 - ReLU-Aktivierung (statt Sigmoid/tanh).
 - Data Augmentation.
 - Dropout zur Regularisierung.
 - Ensemble aus 7 Netzen.

VGGNet (Simonyan & Zisserman, 2014)

- Idee: konsequent kleine Filter 3×3, viele Schichten.
- Sehr tief (16–19 Layer).
- Hohe Genauigkeit, aber extrem viele Parameter (138 Mio).

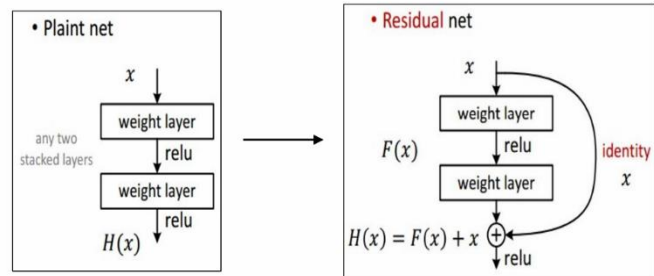
GoogLeNet / Inception (Szegedy et al., 2014)

- Einführung des **Inception-Modules** (mehrere Filtergrößen parallel) (rechts).
- Wenige Parameter (~5 Mio), keine FC-Schichten.
- Sieger bei ILSVRC 2014.



ResNet (He et al., 2015)

- Einführung von **Residual Connections** („skip connections“).
- Ermöglicht extrem tiefe Netze (bis >150 Layer).
- Sieger ILSVRC 2015 (3,6 % Fehler).
- Schlüsselideen: Batch Normalization, He-Initialisierung.

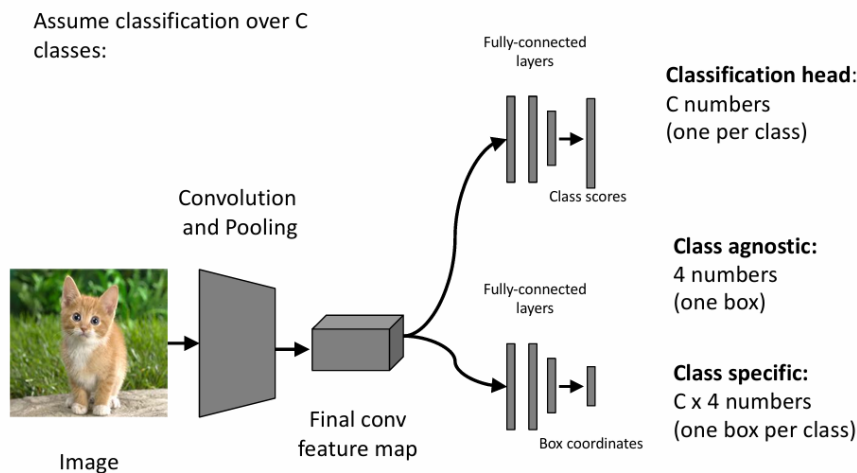


Bildverarbeitung

CNNs lassen sich nicht nur für Klassifikation, sondern auch für **Lokalisierung**, **Objekterkennung** und **Segmentierung** einsetzen.

Klassifikation: Input ist ein Bild, Output ein Klassenlabel, die genutzte Metrik ist Accuracy.

Lokalisierung: Input ist Bild, Output ist eine Box im Bild (x,y,w,h), die genutzte Metrik ist Intersection over Union.



- CNN + Regressionskopf für Bounding Boxes, Loss ist die L2 Distanz zwischen vorhergesagten und wahren Box-Koordinaten.
- Beispiel OverFeat (Kombination von Klassifikation und Regressionsnetz) mit Sliding Window Verfahren → Effizienzsteigerung durch Umwandeln von FC-Layern in Convolutions. Greedy Merging der Boxen nach Sliding Window Verfahren.



Objekterkennung

Mehrere Objekte in einem Bild, Output sind mehrere Bounding Boxes und die zugehörigen Klassenlabel, die genutzte Metrik ist mean Average Precision (mAP).

Detection als Klassifizierung hat das Problem, dass man viele Positionen und Skalierungen betrachten muss. Wenn der Klassifizierer schnell genug ist sollte man es einfach so machen. (Für mehr Informationen Deformable Parts Model anschauen). Wenn der Klassifizierer nicht schnell genug ist ein kleines Subset von möglichen Positionen betrachten.

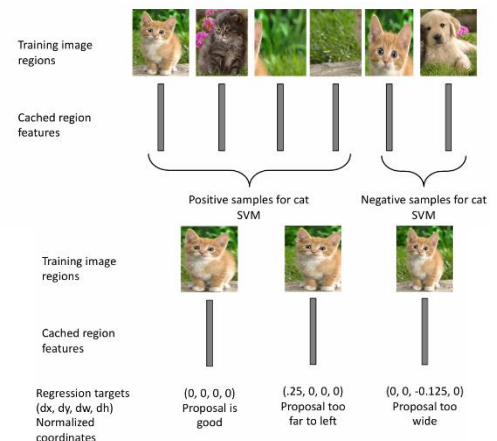
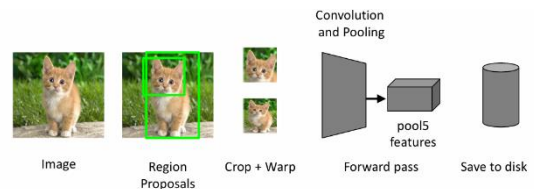
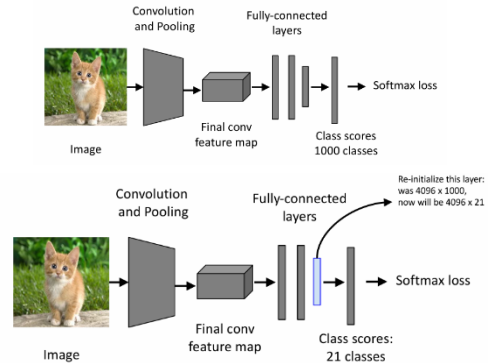
R-CNN

Region-Proposals: Finden Sie „klecksige“ Bildbereiche, die wahrscheinlich Objekte enthalten
„Klassenunabhängiger“ Objektdetektor.

- **Selective Search:** Bottom-up-Segmentierung, Zusammenführung von Regionen auf mehreren Ebenen. Regionen in Boxen transformieren.

Training:

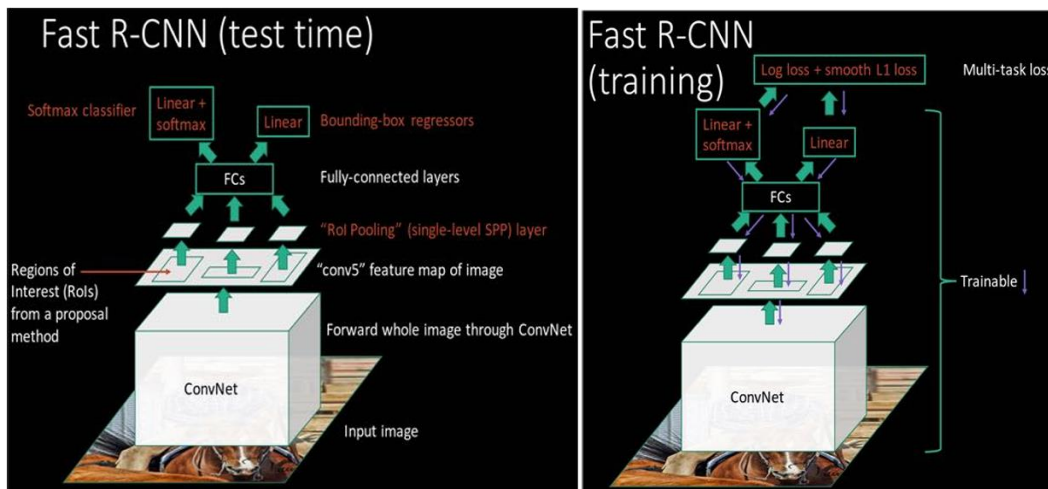
1. Trainieren (oder Downloaden) eines Klassifizierungsmodells für ImageNet (AlexNet)
2. Model für Erkennung Fine-tunen
 - a. Anstelle von 1000 ImageNet-Klassen sollen 20 Objektklassen + Hintergrund verwendet werden.
 - b. Die letzte vollständig verbundene Schicht wird entfernt und von Grund auf neu initialisiert.
 - c. Das Trainingsmodell wird unter Verwendung positiver/negativer Bereiche aus Erkennungsbildern weiter trainiert.
3. Features extrahieren
 - a. Extrahieren Sie Bereichsvorschläge für alle Bilder.
 - b. Für jeden Bereich: Auf CNN-Eingabegröße verzerren, durch CNN vorwärts ausführen, pool5-Merkmale auf Festplatte speichern.
 - c. Verwenden Sie eine große Festplatte: Die Merkmale haben für den PASCAL-Datensatz eine Größe von ~200 GB!
4. Trainieren Sie eine binäre SVM pro Klasse, um Regionsmerkmale zu klassifizieren.
5. (bbox-Regression): Trainieren Sie für jede Klasse ein lineares Regressionsmodell, um die zwischengespeicherten Merkmale den Offsets und GT-Boxen zuzuordnen, um „leicht falsche“ Vorschläge auszugleichen.



Probleme:

- Langsam bei der Testausführung: Für jeden Regionsvorschlag muss ein vollständiger Vorwärtsthroughlauf des CNN durchgeführt werden.
- SVMs und Regressoren sind nachträglich: CNN-Merkmale werden nicht als Reaktion auf SVMs und Regressoren aktualisiert
- Komplexe mehrstufige Trainingspipeline

Fast R-CNN



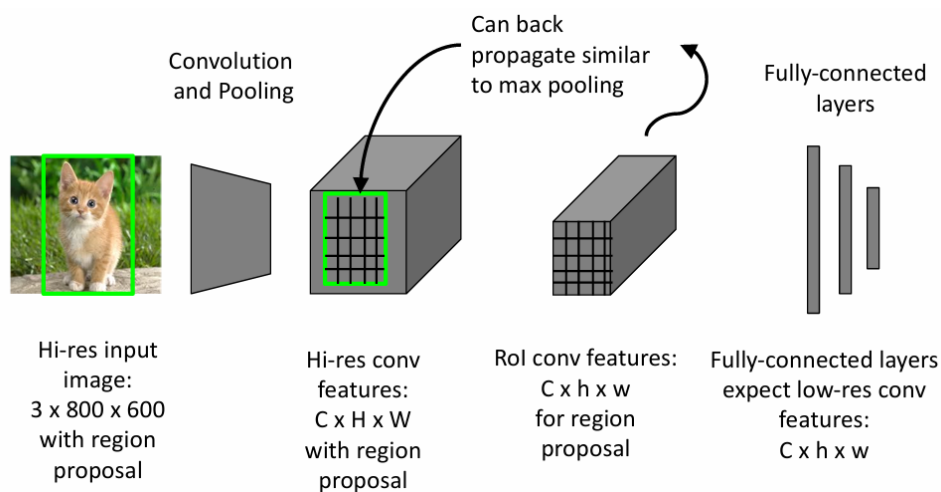
Verbesserungen:

- Gemeinsame Berechnung der Convolutions für das ganze Bild.
- **Region of Interest (RoI) Pooling:** projiziert Regionen auf die Feature Map.
- End-to-End Training möglich.

Vorteile:

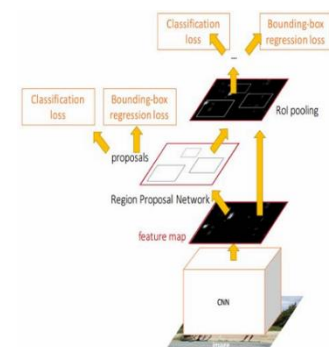
- Deutlich schneller (84h → 9.5h Training; 47s → 0.32s Test).
- Einfache Pipeline.

Region of Interest Pooling:



Faster R-CNN

- Einführung des **Region Proposal Networks (RPN)**.
- CNN generiert selbst Regionsvorschläge.
- Training: gemeinsames Optimieren von Klassifikations- und Regressionsloss.
- Ergebnis: Echtzeitfähigkeit, mAP vergleichbar mit Fast R-CNN, aber 250× schneller.



YOLO

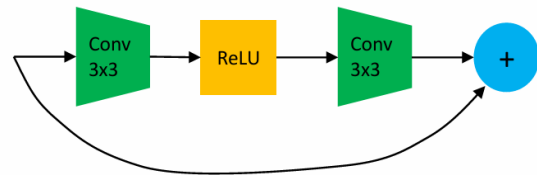
- Ansatz: „You Only Look Once“.
- Bild in S×S-Grid aufgeteilt.
- Jedes Grid-Cell sagt vorher:
 - B Bounding Boxes,
 - Konfidenzwerte,
 - Klassenscores.
- Vorteile: extrem schnell (Echtzeit).
- Nachteil: etwas geringere Genauigkeit als Faster R-CNN.

Evaluation

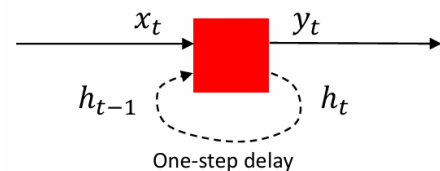
- Standard-Metrik: **mean Average Precision (mAP)**.
- Definiert als Mittelwert der Average Precision pro Klasse.
- Ein Detektion gilt als korrekt, wenn IoU ≥ 0.5 mit einem Ground Truth Objekt.

Recurrent Networks

Standard-Neuronale Netze, wie sie in vorherigen Kapiteln behandelt wurden, sind **gerichtete azyklische Graphen (DAGs)**. Sie verarbeiten Eingaben schichtweise, jede Eingabe wird als unabhängige Instanz betrachtet. Dies ist geeignet für Aufgaben wie Bildklassifikation, aber unzureichend für Daten mit **zeitlicher oder sequentieller Abhängigkeit** (z. B. Sprache, Texte, Zeitreihen, Video).



Recurrent Neural Networks (RNNs) erweitern neuronale Netze um **Rückkopplungen (Zyklen)**. Damit können sie Informationen über Zeit hinweg speichern und sind in der Lage, ganze Sequenzen $x_1, x_2, \dots, x_T$ zu verarbeiten und Sequenzen von Ausgaben $y_1, y_2, \dots, y_T$ zu erzeugen.



Ein RNN enthält einen **Hidden State h_t** , der Informationen aus vorherigen Zeitschritten speichert. Wir können eine Folge von Vektoren x verarbeiten, indem wir bei jedem Zeitschritt eine Rekursionsformel anwenden:

$$h_t = f_W(h_{t-1}, x_t)$$

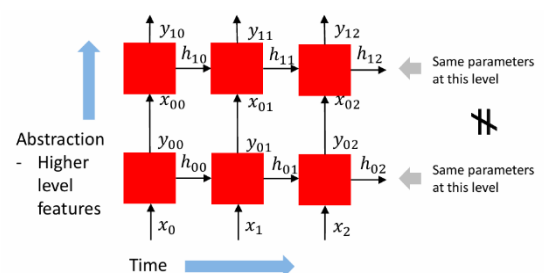
$$h_t = \tanh(W_{hh}h_{t-1} + W_{hx}x_t)$$

$$y_t = W_{hy}h_t$$

Der gleiche Parametersatz W wird zu jedem Zeitpunkt wiederverwendet.

Unrolling

Durch „Unrolling“ über T Zeitschritte ergibt sich ein DAG, auf dem Backpropagation anwendbar ist (**Backpropagation Through Time, BPTT**). Ein RNN wird für jedes Zeitschritt explizit dargestellt. Gradienten können wie in Feedforward-Netzen berechnet werden. Aber Graphgröße wächst mit Sequenzlänge → hoher

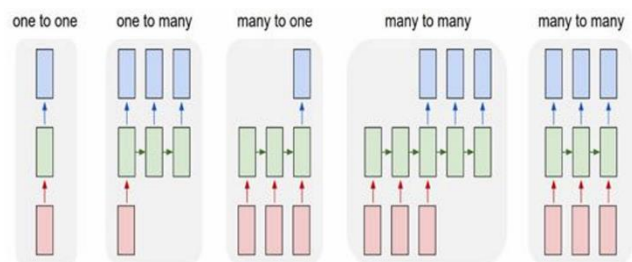


Speicher- und Rechenaufwand. Bei tiefen RNNs werden mehrere RNN-Schichten vertikal gestapelt → höhere Abstraktion und komplexere Repräsentationen.

Training

- **Backpropagation Through Time (BPTT):**
 - Vorwärtsdurchlauf: Speicherung aller Zwischenzustände.
 - Rückwärtsdurchlauf: Anwendung der Kettenregel über alle Zeitschritte.
 - Problematisch: Gradienten können **verschwinden (vanishing gradient)** oder **explodieren (exploding gradient)**.
- **Gradientenprobleme:**
 - Explodierende Gradienten: können durch **Gradient Clipping** kontrolliert werden.
 - Verschwindende Gradienten: erschweren das Lernen von Langzeitabhängigkeiten.

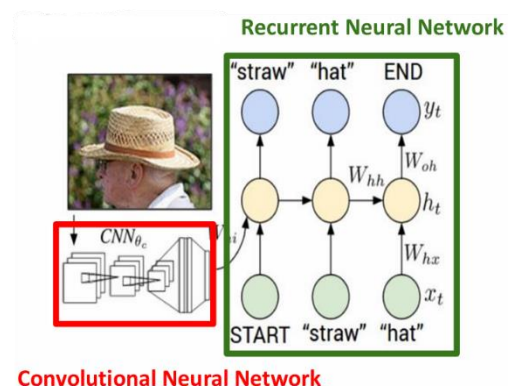
RNNs können sehr unterschiedliche Input-/Output-Mappings abbilden:



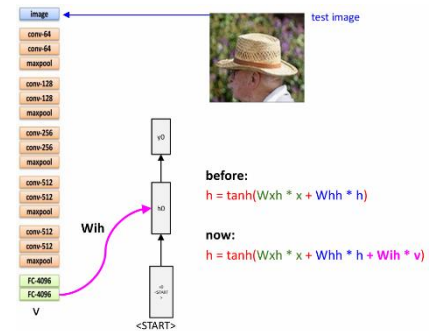
- **One-to-Many:** z. B. Bildbeschreibung (ein Bild → Sequenz von Wörtern).
- **Many-to-One:** z. B. Sentiment-Analyse (Sequenz von Wörtern → Klassenlabel).
- **Many-to-Many (gleichlang):** z. B. Video-Frame-Klassifikation (Frame-Sequenz → Label-Sequenz).
- **Many-to-Many (variabel):** z. B. maschinelle Übersetzung (Wortsequenz → Wortsequenz).

Anwendungen

- Sprachmodellierung (Character- oder Word-Level).
- Funktionsweise: RNN liest Zeichen für Zeichen, Hidden State speichert Kontext, Ausgabe-Vektor beschreibt Wahrscheinlichkeit für nächstes Zeichen.
- Training: Minimierung des Cross-Entropy Loss zwischen vorhergesagtem und tatsächlichem nächsten Zeichen.
- Maschinelle Übersetzung (Encoder-Decoder-Architekturen).
- Bildunterschriften (Kombination CNN + RNN).
 - Erklären von Bildern mit multimodalen rekurrenten neuronalen Netzen, Mao et al.
 - Tiefe visuell-semantische Ausrichtungen zur Erzeugung von Bildbeschreibungen, Karpathy und Fei-Fei
 - Zeigen und erzählen: Ein neuronaler Bildbeschriftungsgenerator, Vinyals et al.



- Langfristige rekurrenten Faltungsnetzwerke für visuelle Erkennung und Beschreibung, Donahue et al.
- Erlernen einer rekurrenten visuellen Darstellung für die Erzeugung von Bildbeschriftungen, Chen und Zitnick
- Videoanalyse (z. B. Frame-Sequenzen).
- Bildgenerierung (z. B. DRAW-Modell).
- Aufmerksamkeit (Attention Mechanisms) → Verarbeitung komplexer Sequenzen mit Fokus.



Long Short-Term Memory (LSTM)

Vanilla-RNNs können nur begrenzte zeitliche Abhängigkeiten lernen, da Gradienten verschwinden oder explodieren. LSTMs (Hochreiter & Schmidhuber, 1997) wurden entwickelt, um dieses Problem zu lösen.

Eine LSTM-Zelle hat zusätzlich zum Hidden State h_t einen **Cell State** c_t , der Informationen langfristig speichert. Gates steuern den Informationsfluss:

- **Forget Gate:** entscheidet, welche Teile von c_{t-1} gelöscht werden.
- **Input Gate:** entscheidet, welche neuen Informationen in den Cell State aufgenommen werden.
- **Output Gate:** entscheidet, welche Teile von c_t an den Hidden State h_t und die Ausgabe weitergegeben werden.

Durch diese additive Struktur im Cell State wird das Vanishing-Gradient-Problem erheblich abgeschwächt.

Vanilla RNN:

$$h_t^l = \tanh(W^l \begin{pmatrix} h_{t-1}^l \\ x_t \end{pmatrix})$$

$h \in \mathbb{R}^n$ $W^l [n \times 2n]$

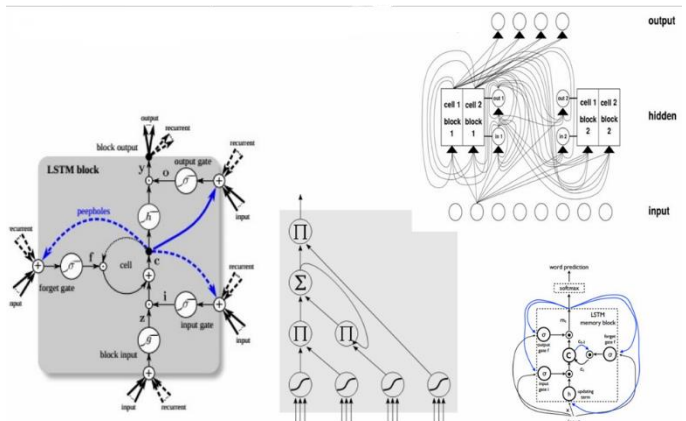
LSTM:

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \text{sigm} \\ \text{sigm} \\ \text{sigm} \\ \text{tanh} \end{pmatrix} W^l \begin{pmatrix} h_{t-1}^l \\ x_t \end{pmatrix}$$

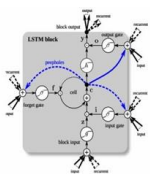
$$c_t^l = f \odot c_{t-1}^l + i \odot g$$

$$h_t^l = o \odot \tanh(c_t^l)$$

$W^l [4n \times 2n]$



Varianten:



[LSTM: A Search Space Odyssey, Greff et al., 2015]

GRU [Learning phrase representations using rnn encoder-decoder for statistical machine translation, Cho et al. 2014]

$$\begin{aligned} r_t &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\ z_t &= \text{sigm}(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \\ \tilde{h}_t &= \tanh(W_{xh}x_t + W_{hh}(r_t \odot h_{t-1}) + b_h) \\ h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \end{aligned}$$

MUT1:

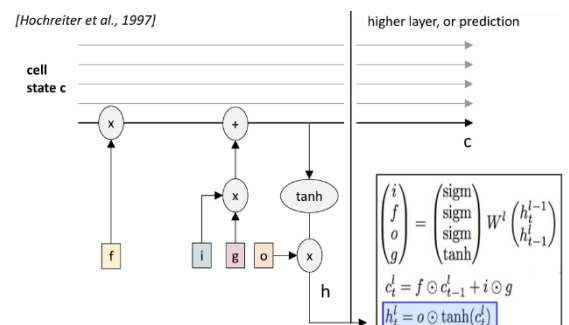
$$\begin{aligned} z &= \text{sigm}(W_{xz}x_t + b_z) \\ r &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\ h_{t+1} &= \tanh(W_{zh}(r \odot h_t) + \tanh(x_t) + b_h) \odot z + h_t \odot (1 - z) \end{aligned}$$

MUT2:

$$\begin{aligned} z &= \text{sigm}(W_{xz}x_t + W_{zh}h_{t-1} + b_z) \\ r &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\ h_{t+1} &= \tanh(W_{zh}(r \odot h_t) + W_{zh}x_t + b_h) \odot z + h_t \odot (1 - z) \end{aligned}$$

MUT3:

$$\begin{aligned} z &= \text{sigm}(W_{xz}x_t + W_{zh}(\tanh(h_t) + b_z)) \\ r &= \text{sigm}(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \\ h_{t+1} &= \tanh(W_{zh}(r \odot h_t) + W_{zh}x_t + b_h) \odot z + h_t \odot (1 - z) \end{aligned}$$



- **GRU (Gated Recurrent Unit, Cho et al. 2014):** vereinfachte Version der LSTM mit nur zwei Gates (Reset, Update). Weniger Parameter, vergleichbare Performance.
- **Bidirektionale RNNs:** zwei RNNs laufen in Vorwärts- und Rückwärtsrichtung, geeignet für Aufgaben, bei denen Kontext aus Vergangenheit und Zukunft nützlich ist.

- **Stacked RNNs:** mehrere RNN-Schichten übereinander für komplexere Repräsentationen.

Gradientendynamik:

- Gradientendynamik hängt von den Eigenwerten der Gewichtsmatrix W_{hh} ab.
 - Größter Eigenwert $> 1 \rightarrow$ Gradienten explodieren.
 - Größter Eigenwert $< 1 \rightarrow$ Gradienten verschwinden.
- **Kontrolle:**
 - Exploding Gradients $\rightarrow$ Clipping.
 - Vanishing Gradients $\rightarrow$ LSTMs oder GRUs mit additiven Zuständen.

Generative Models

Generative Modelle gehören zum Bereich des **Unsupervised Learning**. Während beim überwachten Lernen ein Mapping von Eingabedaten x auf Labels y gelernt wird, stehen beim unüberwachten Lernen lediglich Daten x ohne Labels zur Verfügung. Ziel ist es, verborgene Strukturen und Wahrscheinlichkeitsverteilungen in den Daten zu modellieren.

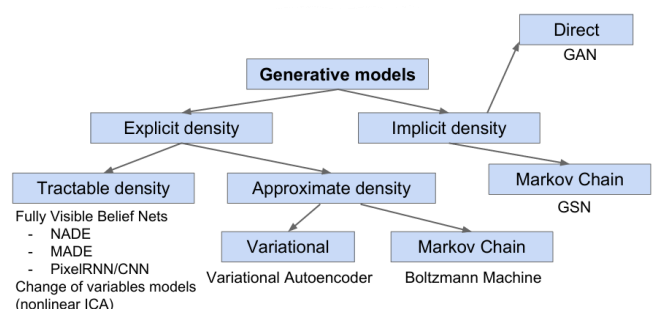
Generative Modelle haben die Aufgabe, eine **Modellverteilung** $p_{\text{model}}(x)$ zu lernen, die der wahren Datenverteilung $p_{\text{data}}(x)$ möglichst ähnlich ist. Damit können neue, realistisch wirkende Datenpunkte generiert werden.

Anwendungen:

- Bildsynthese, Super-Resolution, Farbgebung, Kunst.
- Simulation und Planung in Zeitreihen (z. B. Reinforcement Learning).
- Lernen von latenten Repräsentationen für nachgelagerte Aufgaben.

Generative Modelle lassen sich grob in zwei Kategorien einteilen:

- **Explizite Dichte-Modelle:** Modellieren $p(x)$ direkt.
 - Tractable density: Likelihood kann exakt berechnet werden (z. B. PixelRNN, PixelCNN).
 - Approximate density: Likelihood ist intractable, wird aber approximiert (z. B. Variational Autoencoders).
- **Implizite Dichte-Modelle:** Kein explizites $p(x)$, stattdessen Sampling über ein Generationsverfahren (z. B. GANs).



Pixel RNN/CNN

„Fully Visible Belief Networks“ – Likelihood des Bildes wird mithilfe der Kettenregel in Produkt bedingter Wahrscheinlichkeiten zerlegt:

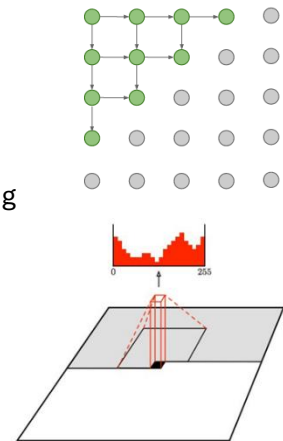
$$p(x) = \prod_{i=1}^n p(x_i | x_1, \dots, x_{i-1})$$

Jeder Pixel hängt von allen vorherigen Pixeln ab. Training: Maximierung der Likelihood der Trainingsdaten.

PixelRNN: Nutzung von RNNs (insb. LSTMs), um Abhängigkeiten von vorherigen Pixeln zu modellieren. Generierung sequenziell Pixel für Pixel → sehr realistische, aber langsame Generierung.

PixelCNN: Abhängigkeiten werden mithilfe von Convolutions über den Kontextbereich modelliert. Training schneller als PixelRNN (Parallelisierung durch Convolutions). Generierung weiterhin sequentiell, also ebenfalls langsam.

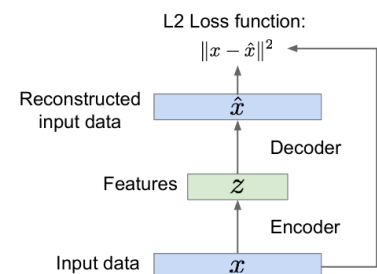
PixelCNN++: Verbesserungen: Gated Convolutions, Shortcut-Verbindungen, diskretisierte logistische Verlustfunktion und Multiscale-Modelle. Likelihood explizit berechenbar, gute Metrik zur Evaluation aber sequentielle Generierung limitiert Geschwindigkeit.



Variational Autoencoders (VAEs)

Autoencoders

- Encoder $f_{\phi}(x) \rightarrow$ latente Repräsentation z .
- Decoder $g_{\theta}(z) \rightarrow$ Rekonstruktion $\hat{x}$.
- Training: Minimierung des Rekonstruktionsfehlers (z. B. L2-Loss).
- Autoencoder lernen latente Faktoren der Variation, können zur Feature-Extraktion und Initialisierung genutzt werden.



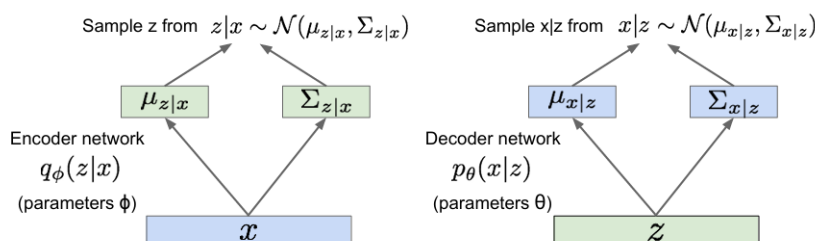
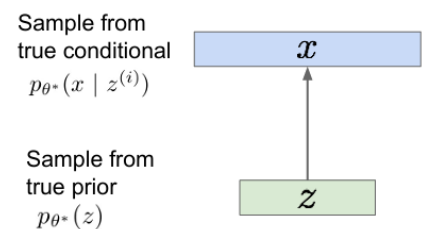
VAEs definieren eine schwer zu handhabende Dichtefunktion mit latentem z :

$$p_{\theta}(x) = \int p_{\theta}(z)p_{\theta}(x|z)dz$$

Kann nicht direkt optimiert werden, leiten Sie stattdessen die untere Grenze der Wahrscheinlichkeit ab und optimieren Sie diese.

Probabilistischer Ansatz bei Autoencodern – ermöglicht es uns, aus dem Modell zu sampeln, um Daten zu generieren!

Angenommen, Trainingsdaten $\{x^{(i)}\}_{i=1}^N$ werden aus einer zugrunde liegenden, nicht beobachtbaren (latenten) Darstellung z generiert.



Wir möchten die wahren Parameter θ^* dieses generativen Modells schätzen. Wählen Sie $p(z)$ einfach, z. B. Gauß. Bedingtes $p(x|z)$ ist komplex (erzeugt Bild) => Darstellung mit neuronalem Netzwerk

Problem Intractable Likelihood: Likelihood-Berechnung erfordert Integration über alle z : intractable. Lösung: Einführung eines zusätzlichen **Encoder-Netzwerks** $q_\phi(z|x)$ zur Approximation des Posteriors.

Optimierung: Maximieren der variationalen unteren Schranke (Evidence Lower Bound, ELBO):

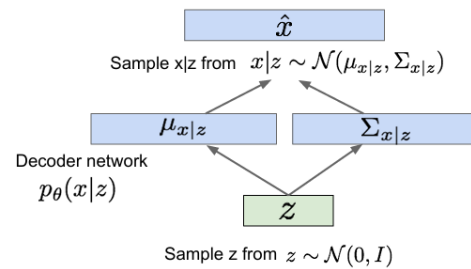
$$\log p_\theta(x) \geq \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - KL(q_\phi(z|x) || p(z))$$

Vorteile von VAEs:

- Prinzipieller probabilistischer Ansatz.
- Liefert latente Repräsentationen, die für andere Aufgaben genutzt werden können.

Nachteile von VAEs:

- Optimierung maximiert nur eine Schranke, nicht die exakte Likelihood.
- Generierte Samples oft unscharf (blurrier) im Vergleich zu GANs.

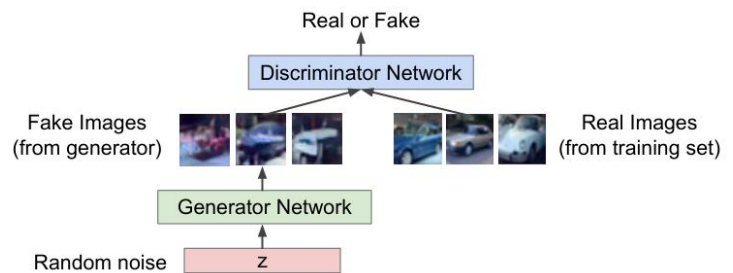


Generative Adversarial Networks (GANs)

Explizite Modellierung von Wahrscheinlichkeitsdichten ist schwierig. GANs verzichten auf explizite Likelihood und lernen stattdessen ein **Sampling-Verfahren**.

Generator $G(z)$: transformiert Zufallsvektor $z \sim p(z)$ (z. B. Gaußrauschen) in synthetische Daten.

Discriminator $D(x)$: klassifiziert Daten als „real“ oder „fake“.



Training

GANs zu trainieren ist ein Zwei-Spieler Spiel, bei dem man gemeinsam eine Minimax Zielfunktion trainiert. Optimierung erfolgt abwechselnd für D und G .

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Der Diskriminator (θ_d) möchte das Ziel maximieren, sodass $D(x)$ nahe bei 1 (echt) und $D(G(z))$ nahe bei 0 (falsch) liegt.

Generator (θ_g) möchte das Ziel minimieren, sodass $D(G(z))$ nahe bei 1 liegt (der Diskriminator wird dazu gebracht, zu glauben, dass das generierte $G(z)$ echt ist).

Training instabil, Gradienten können verschwinden.

Vorteile: erzeugt realistischste Samples aller Ansätze.

Nachteile: schwierig zu trainieren, keine Inferenz $p(x)$ oder $p(z|x)$.

Algorithmus:

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

Deep Probabilistic Models

Neben rein neuronalen Ansätzen spielen **probabilistische Modelle** eine zentrale Rolle in der KI. Sie ermöglichen explizite Modellierung von Unsicherheit und Abhängigkeiten zwischen Variablen. Graphical Models (GMs) bilden eine Brücke zwischen Statistik, Wahrscheinlichkeitstheorie und maschinellem Lernen. Sie sind eine kompakte, graphische Notation für bedingte Unabhängigkeiten und damit effiziente Repräsentationen von Wahrscheinlichkeitsverteilungen.

Graphical Models sind in vielen Bereichen präsent:

- **Spracherkennung:** Hidden Markov Models (HMMs) → Sequenzen von Lauten zu Wörtern.
- **Bildverarbeitung:** Markov Random Fields (MRFs) → z. B. Bildentrauschen.
- **Breite Anwendungen:** Information Retrieval, Suchmaschinen, Collaborative Filtering, Bioinformatik, Genexpressionsanalyse, NLP.

Grundlagen

Diskrete Zufallsvariablen sind Variablen, die nur endlich viele oder abzählbar unendlich viele Werte annehmen können, und sie sind entscheidend für die Wahrscheinlichkeitsrechnung. Zustandsraum $X = \{x_1, \dots, x_n\}$ und Wahrscheinlichkeitsraum $P(X = x_i) \geq 0, \sum_i P(X = x_i) = 1$

Die **gemeinsame Wahrscheinlichkeitsfunktion (JPF)** ist ein grundlegendes Konzept in der Wahrscheinlichkeitstheorie und Statistik. Sie stellt die Wahrscheinlichkeit dar, dass zwei oder mehr Zufallsvariablen gleichzeitig auftreten. Die JPF ist besonders nützlich in Szenarien, in denen die Interaktion zwischen Variablen entscheidend ist, wie beispielsweise bei der multivariaten Analyse. Die JPF wird mathematisch als $P(X, Y)$ ausgedrückt, wobei X und Y zwei diskrete Zufallsvariablen sind. Diese Notation gibt die Wahrscheinlichkeit an, dass X einen bestimmten Wert annimmt, während Y gleichzeitig einen anderen Wert annimmt. Für kontinuierliche Zufallsvariablen wird die gemeinsame Wahrscheinlichkeitsdichtefunktion (JPF) durch eine gemeinsame Wahrscheinlichkeitsdichtefunktion (PDF) dargestellt, die als $f(x, y)$ bezeichnet wird. Enthält vollständige Information über Abhängigkeiten, aber **exponentiell groß** in der Anzahl der Variablen.

Operationen:

- Marginalisierung: $P(X) = \sum_Y P(X, Y)$

- Produktregel: $P(X, Y) = P(X|Y)P(Y)$
- Bayes Regel: $P(X|Y) = \frac{P(X|Y)P(Y)}{P(Y)}$

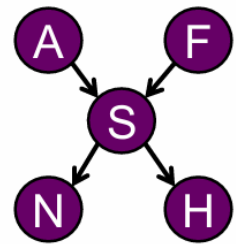
Stochastische Unabhängigkeit beschreibt zwei Ereignisse A und B, bei denen das Eintreten des einen Ereignisses keinen Einfluss auf die Wahrscheinlichkeit des anderen Ereignisses hat. Das bedeutet, dass die Wahrscheinlichkeit von A und B unabhängig voneinander ist. $P(A, G) = P(A)P(G)$

Bedingte Unabhängigkeit beschreibt eine Situation, in der zwei Ereignisse A und B unabhängig sind, wenn ein drittes Ereignis C bekannt ist. Das bedeutet, dass die Wahrscheinlichkeit von A gegeben C gleich der Wahrscheinlichkeit von A ist, unabhängig von B. $P(C|S, G, A) = P(C|S)$

Graphische Modelle

Bayesian Networks (BNs)

DAG-Struktur: Knoten = Zufallsvariablen, Kanten = Abhängigkeiten. Jede Variable X_i erhält eine bedingte Verteilung $P(X_i|Pa(X_i))$.



Joint Distribution: $P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i|Pa(X_i))$

Lokale Markov-Annahme: jede Variable ist unabhängig von Nicht-Nachfolgern, gegeben den Eltern.

Inferenz

Berechnung von Wahrscheinlichkeiten wie $P(X|E=e)$.

Variable Elimination: Gegeben sei ein BN und eine Abfrage $P(X|e) / P(X,e)$ Instanzieren Sie den Beweis e. Wählen Sie eine Eliminierungsreihenfolge für die Variablen, z. B. $X_1, \dots, X_n$. Initialisieren Sie die Faktoren $\{f_1, \dots, f_n\}$: $f_i = P(X_i|Pa(X_i))$ (CPT für X_i).

Für $i = 1$ bis n , wenn $X_i \notin \{X, E\}$

- Sammeln Sie die Faktoren $f_1, \dots, f_k$, die X_i enthalten.
- Generieren Sie einen neuen Faktor, indem Sie X_i aus diesen Faktoren eliminieren. $g = \sum_{X_i} \prod_{j=1}^k f_j$
- Die Variable X_i wurde eliminiert! Fügen Sie g zum Satz der Faktoren hinzu.

Normalisieren Sie $P(X,e)$, um $P(X|e)$ zu erhalten.

Komplexität: exponentiell in der Baumweite (treewidth) des Graphen.

Inferenz ist im Allgemeinen **#P-vollständig (NP-hard)**.

Markov Networks

Ungerichtete Graphen, Faktorisierung über Cliques.

Faktorgraphen

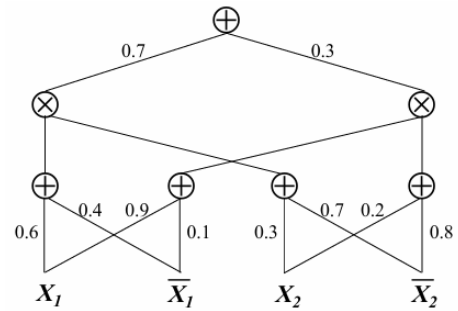
Bipartite Repräsentation: Variablen- und Faktor-Knoten.

Sum-Product Networks

SPNs sind DAGs mit Knoten für **Summen**, **Produkte** und **Indikatoren**. Die Gewichte auf Kanten von Summenknoten repräsentieren Mischungskoeffizienten.

Semantik:

- **Summe:** Mischung (Marginalisierung über latente Variablen).
- **Produkt:** Merkmalskombination (hierarchische Faktorisierung).



Inference: linear in der Netzgröße.

- Wahrscheinlichkeiten und Marginals können durch Vorwärtsauswertung berechnet werden.
- MAP-Inferenz: Ersetzen von Summen durch Max-Operationen.

Learning: durch EM oder Gradientenverfahren.

Vorteile:

- Kompakte Repräsentationen.
- Tiefe Architekturen mit vielen Schichten möglich.
- Traktable Inferenz auch für große Modelle.

Anwendungen:

- **Bildvervollständigung:** SPNs outperformen DBNs und DBMs in Geschwindigkeit und Genauigkeit.
- **Ausreißererkennung:** SPNs wissen „wann sie etwas nicht wissen“.
- **Zeitreihenmodellierung:** Erweiterungen wie Whittle Networks.
- **NLP:** Poisson-SPNs für Topic Models.
- **Kryptographie:** Crypto-SPNs ermöglichen abgesicherte Berechnungen.
- **Hybrid AI:** Integration von SPNs mit neuronalen Netzen, z. B. für Szene- und Physiklernen.

Tractable Probabilistic Models

Tractable Probabilistic Models (TPMs) sind probabilistische Modelle, die **exakte Inferenz in polynomieller Zeit** erlauben.

Im Zentrum steht der Zielkonflikt:

- **Expressivität:** Fähigkeit, komplexe Verteilungen kompakt zu repräsentieren.
- **Traktabilität:** Möglichkeit, wichtige Inferenzanfragen effizient zu beantworten.

Probabilistic Circuits (SPNs, ACs, PSDDs, CNet) bilden einen einheitlichen Rahmen, um diese Eigenschaften zu verbinden.

Tractable Probabilistic Inference

Eine Anfrageklasse Q ist auf einer Modellfamilie M **traktabel**, wenn jede Anfrage $q \in Q$ auf einem Modell $m \in M$ exakt in $O(\text{poly}(|q| \cdot |m|))$ berechnet werden kann.

Anfragenklassen:

- **EVI (Complete Evidence Inference):** Wahrscheinlichkeit einer vollständigen Zuweisung.
- **MAR (Marginal):** Wahrscheinlichkeit einer partiellen Zuweisung.
- **CON (Conditional):** bedingte Wahrscheinlichkeiten.
- **MAP/MMAP:** wahrscheinlichste Erklärungen (NP-schwer für viele Modelle).
- **Advanced Queries:** logische Events, Gruppierungen, Erwartungswerte.

Limitationen von Graphischen Modellen:

- **Bayesian Networks (BNs):** faktorisierbar, EVI tractable.
- **Markov Networks (MNs):** Normalisierungsfaktor $Z \rightarrow$ EVI intractable.
- **Treewidth:** bestimmt Komplexität. Exakte MAR/CON in $O(|X| \cdot 2^w)$.

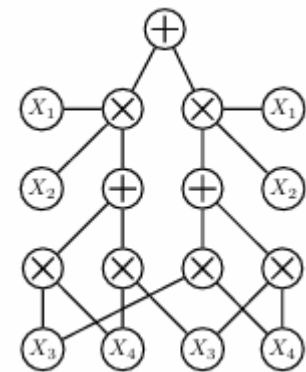
Tractability vs Expressivity

- Vollständig faktorisierte Modelle: sehr effizient, kaum expressiv.
- Mixturen: erhöhte Expressivität, lineare Inferenz in Anzahl der Komponenten.
- VAEs, GANs, NFs: sehr expressiv, aber nicht tractable.
- **Probabilistic Circuits:** an der „Sweet Spot“-Position zwischen beiden Extremen.

Probabilistic Circuits

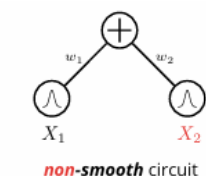
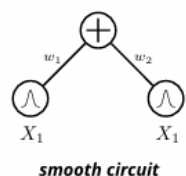
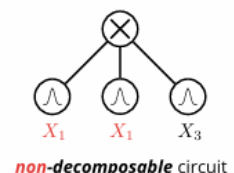
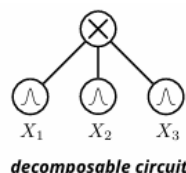
Probabilistic Circuits sind DAGs mit **Summen-, Produkt- und Verteilungsknoten**, wobei Summe = Mischung, Produkt = Faktorisierung. Die Semantik von PCs ist rekursiv definierbar.

Im Vergleich zu PGMs sind die Knoten keine Zufallsvariablen sondern Rechenoperationen. Außerdem sind sie differenzierbar und daher mit Gradientenverfahren trainierbar.



Strukturbedingungen:

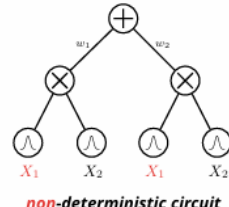
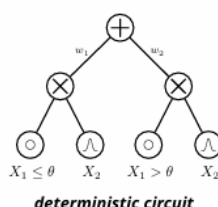
- **Decomposability:** Kinder eines Produkts disjunkte Variablen.
- **Smoothness:** Kinder einer Summe gleiche Variablenmengen.
- **Determinism:** nur ein aktives Kind pro Summe.



Diese Bedingungen sichern Traktabilität für MAR/CON/MAP.

Modelle:

- **Arithmetic Circuits (ACs):** deterministisch, glatt, dekomponierbar.
- **Sum-Product Networks (SPNs):** eingeführt von Poon & Domingos (2011), erlauben effiziente Inferenz und tiefere Hierarchien.
- **Cutset Networks (CNets):** Mischung von Baumstrukturen, ebenfalls kompilierbar.



- **PSDDs (Probabilistic Sentential Decision Diagrams):** logische Struktur + Wahrscheinlichkeiten, ermöglichen komplexe Anfragen.

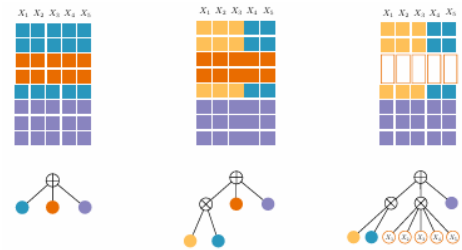
Lernen von Probabilistic Circuits

Parameterlernen

- **Deterministische Circuits:** Maximum Likelihood analytisch berechenbar.
- **Nicht-deterministische Circuits:** EM, SGD, Bayesian Approaches.
- Erweiterungen: oBMM, Collapsed Variational Bayes, CCCP.

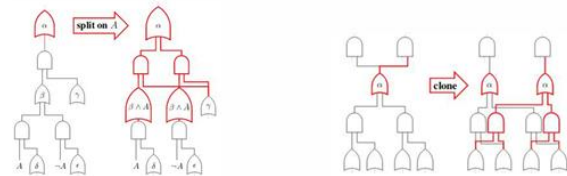
Strukturlernen

- **LearnSPN (Gens & Domingos):** rekursives Splitten von Instanzen und Variablen.
 - Varianten: ID-SPN, DAG-SPNs, RAT-SPNs.
- **PSDD-Lernen:** inkrementelle Strukturänderungen, vtrees.
- **Bayesian Ansätze:** Prior über Strukturen (Dirichlet Process).



Ensembles

- Bagging, Boosting, Randomized Structures → Robustheit und Generalisierung.



Anwendungen

- **Dichteabschätzung:** Benchmarks zeigen oft bessere Log-Likelihoods als BNs, VAEs, MADEs.
- **Bildverarbeitung:** Imputation, Completion, Anomaly Detection.
- **NLP:** Topic Models, Sprachmodelle.
- **Probabilistic Programming:** Kompilierung von PGMs in Circuits.
- **Hybrid AI:** Integration in Deep Learning für Unsicherheitsmodellierung.

Erweiterungen

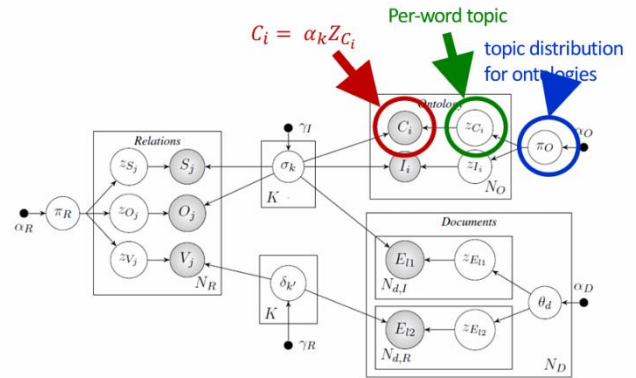
- **Semi-Ring Inferenz:** Nutzung anderer Algebra-Strukturen (z. B. logische Inferenz via Weighted Model Counting).
- **Verknüpfung mit logischen Circuits:** d-DNNFs, OBDDs, SDDs als Vorläufer.
- **RAT-SPNs:** Tensorisierte Strukturen, kompatibel mit Autodiff.
- **ABDA (Automatic Bayesian Density Analysis):** automatische Analyse heterogener Daten.

Deep NLP

Natural Language Processing (NLP) beschäftigt sich mit der automatischen Verarbeitung, Analyse und Generierung natürlicher Sprache. Im Kontext des Deep Learning stehen dabei **semantische Repräsentationen** im Vordergrund: Texte werden in Vektoren eingebettet, die Bedeutung, Struktur und Relationen abbilden.

Propositional Semantics

- Übersetzung von Text in logische Formeln (z. B. Prädikatenlogik).
- Beispiel: „dog bites man“ → bites(dog,man).
- Vorteil: ermöglicht logische Inferenz (z. B. „Alle Menschen sind sterblich“ + „Sokrates ist ein Mensch“ → „Sokrates ist sterblich“).
- Anwendung: DARPA „Big Mechanism“ (biomedizinische Inferenz).
- Moderne Varianten: Latente Variablenmodelle (z. B. KB-LDA) → Entitäten, Relationen und Hierarchien gemeinsam lernen.



Vektorielle Semantik

- Texte in hochdimensionalen Raum einbetten: $vec(satz) \in R^n$
- Semantisch ähnliche Sätze haben ähnliche Vektoren.
- Vorteil: kontinuierliche, datengetriebene Repräsentation; geeignet für Deep Learning.

Wort- und Satzrepräsentationen

Bag-of-Words

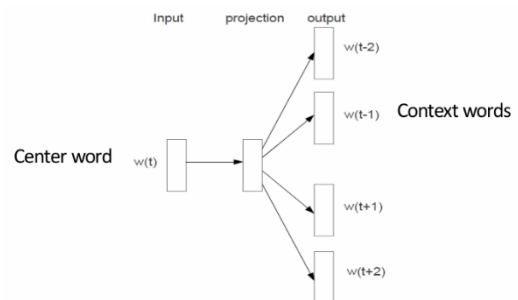
- Dokument als Häufigkeitsvektor (tf-idf).
- Latent Semantic Analysis (LSA): Matrixfaktorisierung $T \approx USV^T$.
- Liefert latente Dimensionen = Themen.
- Limitation: keine Wortreihenfolge, keine Syntax.

Word Embeddings

Verteilte Repräsentationen von Wörtern. Hypothese: „Words that occur in similar contexts have similar meaning“ (paradigmatische Ähnlichkeit).

word2vec (Skip-Gram, CBOW).

- Skip-Gram: Vorhersage von Kontextwörtern aus einem Zentrum.
- CBOW: Vorhersage des Zentrums aus dem Kontext.
- Training über Softmax-Loss: $P(j|i) = \frac{\exp(u_j^T v_i)}{\sum_{k=1} \exp(u_k^T v_i)}$

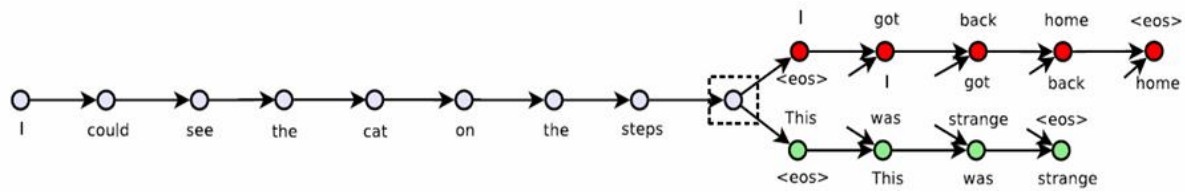


GloVe: Faktorisierung von Wort-Kooccurrenzmatrizen.

Eigenschaften: lineare Algebra-Operationen auf Embeddings repräsentieren semantische Relationen („king - man + woman ≈ queen“).

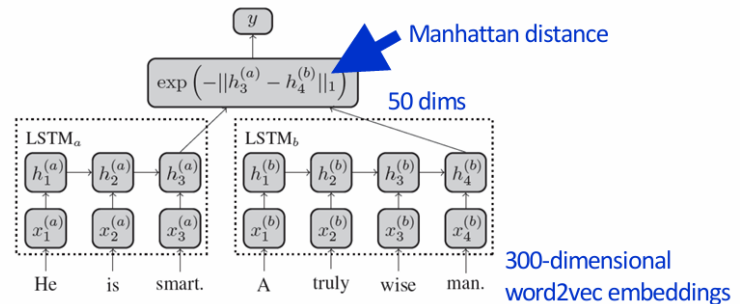
Skip-Thought Vectors

RNN-basiert: Vorhersage des vorherigen und nächsten Satzes. Liefert Satz-Embedding, das semantische und strukturelle Information enthält.



Siamese Netzwerke

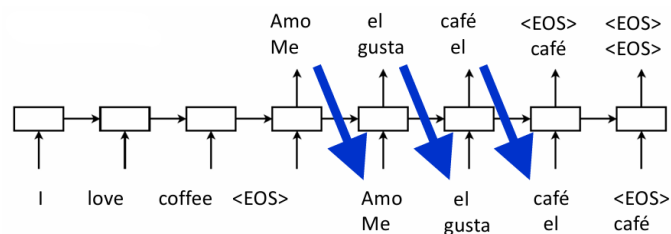
- Eingabe: Satzpaare.
- Ziel: Ähnlichkeit (Semantic Relatedness, Entailment).
- Aufbau: zwei identische RNNs mit geteilten Parametern → Distanzmaß (z. B. Manhattan-Distanz).
- Anwendungen: SICK-Datensatz (semantic relatedness, entailment).



Maschinelle Übersetzung

Sequence-to-Sequence RNNs

- Encoder: liest Quellsequenz und bildet Kontextvektor.
- Decoder: generiert Zielsprache Wort für Wort.
- Training: Maximierung der Wahrscheinlichkeit der Zielsequenz.



BLEU-Score

- Standardmetrik für Übersetzungen: n-Gram-Precision, kombiniert über geometrisches Mittel, inkl. Kürzungsstrafe.
- Unigram-Werte erfassen in der Regel die Angemessenheit.
- Ngram-Werte erfassen in der Regel die Sprachgewandtheit.
- Misst Nähe zu Referenzübersetzungen.

$$\frac{\text{richtige Unigrams im Referenzsatz}}{\text{Unigrams im Testsatz}}$$

$$BLEU = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-\frac{r}{c})} & \text{if } c \leq r \end{cases}$$

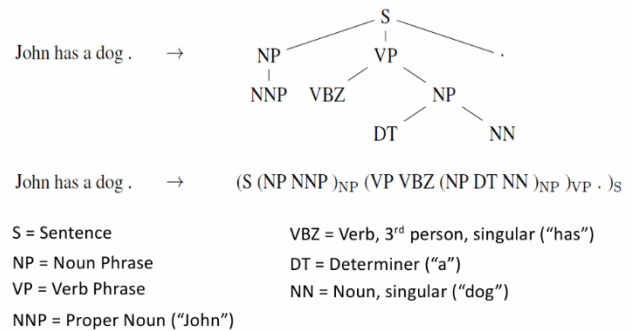
Kandidatenlänge c kürzer als Referenz r Übersetzung

Attention-Mechanismen

- Problem: fester Kontextvektor verliert Information bei langen Sequenzen.
- Lösung: Attention (Bahdanau 2014): gewichtete Summe von Encoder-States abhängig vom Decoder-Zustand.
- Ergebnis: signifikante Verbesserungen in BLEU.

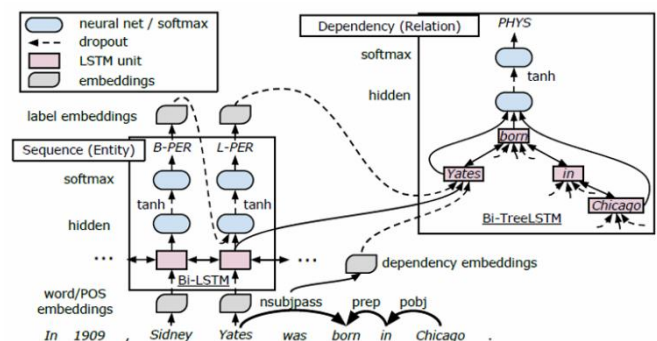
Parsing

- Klassische NLP-Aufgabe: Erzeugung syntaktischer Bäume.
- Sequence-to-Sequence-Parser (Vinyals et al., 2015):
 - Input: Satz.
 - Output: linearisiertes Parse-Tree-Format (z. B. Klammerdarstellung).
 - Mit Attention-Modellen → SOTA-Ergebnisse.
- Besonderheit: Dropout in LSTM-Layern verbessert Generalisierung.



Entity-Relation Extraction

- Ziel: Extraktion von Entitäten und deren Relationen aus Text.
- Ansätze:
 - RNNs direkt auf Text → Schwierigkeiten mit Struktur.
 - Kombination mit Dependenzstrukturen → bessere Ergebnisse.
 - Multi-Input-Modelle: ein RNN für Text, eines für Dependenz-Bäume.



Evaluation

- **Semantic Relatedness:** Vergleich der Nähe zweier Sätze (Skala 1–5).
- **Entailment:** Klassifikation in ENTAILMENT, CONTRADICTION, NEUTRAL.
- Modelle (Skip-Thought, Siamese LSTM) erreichen gute Korrelation mit menschlichen Urteilen.

Transformers

Klassische Sequenzmodelle wie RNNs und LSTMs stoßen an Grenzen:

- Schwierigkeiten mit **langen Abhängigkeiten** (vanishing/exploding gradients).
- Große Anzahl an Trainingsschritten, da **Rekurrenz sequentiell** ist.
- Eingeschränkte **Parallelisierbarkeit**.

Transformers (Vaswani et al., 2017, *Attention is All You Need*) umgehen diese Probleme, indem sie vollständig auf Rekurrenz verzichten und stattdessen **Attention-Mechanismen** einsetzen.

Vorteile:

- Lange Abhängigkeiten modellierbar.
- Keine Probleme mit Gradientenverschwinden/-explosion.
- Weniger Trainingsschritte erforderlich.
- Volle Parallelisierung möglich.

Im Vergleich zu RNNs haben Transformer eine reduzierte Anzahl sequentieller Schritte → vollständig parallelisierbar. Die Pfadlänge zwischen zwei Positionen ist minimal → bessere

Modellierung langer Abhängigkeiten und es ist keine Speicherung von Zuständen über die Zeit notwendig → stabilere Gradienten.

Attention

Attention ahmt eine **Abfrage in einer Datenbank** nach:

- **Query (Q):** Anfragevektor.
- **Keys (K):** Indexvektoren der gespeicherten Daten.
- **Values (V):** gespeicherte Werte.

Formel:

$$attention(q, k, v) = \sum_i similarity(q, k_i) \times v_i$$

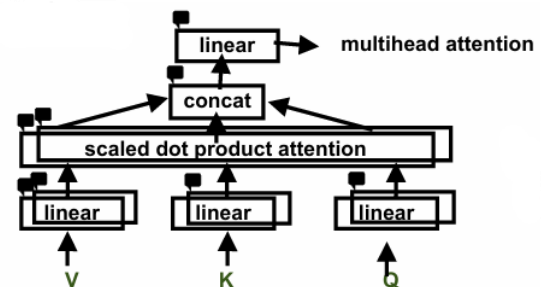
Die Query bestimmt die Gewichtung der Values basierend auf der Ähnlichkeit zwischen Query und Keys.

Architektur

Ein Transformer ist ein **Encoder-Decoder-Modell**, das ausschließlich Attention-Mechanismen nutzt.

Multi-Head-Attention

- Mehrere Attention-Berechnungen parallel („Heads“).
- Unterschiedliche Projektionen von Q, K, V → unterschiedliche Repräsentationen.
- Ergebnis = Konkatenation aller Heads, Projektion zurück in den Modellraum.



Vorteil: Modell kann verschiedene Abhängigkeitsarten gleichzeitig erfassen (analog zu Feature Maps in CNNs).

$$multihead(Q, K, V) = W^O \text{concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_n)$$

$$\text{head}_i = \text{attention}(Q_i^Q Q, W_i^K K, W_i^V V)$$

$$attention(Q, K, V) = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) V$$

Masked Multi-Head Attention

- Speziell im Decoder: zukünftige Positionen werden maskiert.
- Ziel: Vorhersage jedes Tokens darf nur auf **vorherigen Tokens** basieren, nicht auf zukünftigen.
- Realisierung: Maske aus $-\infty$ verhindert Auswahl unerlaubter Positionen.

$$\text{maskedAttention}(Q, K, V) = \text{softmax}\left(\frac{Q^T K + M}{\sqrt{d_k}}\right) V$$

Mit M einer Maskenmatrix aus nullen und $-\infty$.

Weitere Schichten:

- **Layer Normalization:** Normiert Aktivierungen auf Mittelwert 0, Varianz 1 → stabilisiert Training.
 - Für jede versteckte Einheit h_i berechne: $h_i \leftarrow \frac{g}{\sigma} (h_i - \mu)$ wobei g eine Variable, $\mu = \frac{1}{N} \sum_{i=1}^H h_i$ und $\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^H (h_i - \mu)^2}$
- **Feedforward-Netzwerke:** Zwei lineare Transformationen + nichtlineare Aktivierung.
- **Residual Connections:** Addition von Input und Output jeder Sub-Schicht → stabilere Gradienten.
- **Positional Encoding:** Hinzufügen von Positionsinformation (sinus-/cosinus-basiert) zur Unterscheidung der Reihenfolge.
 - $PE_{position,2i} = \sin(\frac{position}{10000^{2i/d}})$
 - $PE_{position,2i+1} = \cos(\frac{position}{10000^{2i/d}})$

Anwendungen

Original Transformer (Vaswani et al. 2017)

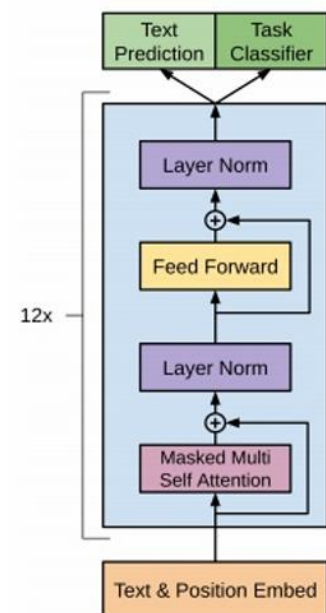
- Aufgaben: Maschinelle Übersetzung (Englisch–Deutsch, Englisch–Französisch).
- Ergebnisse: SOTA bei BLEU-Score, schnelleres Training als RNNs.

GPT und GPT-2 (Radford et al., 2018/2019)

- Reiner **Decoder-Transformer**.
- Aufgabe: **Language Modeling** (nächstes Wort vorhersagen).
- Training: unüberwachtes Pretraining auf großen Korpora.
- Ergebnis: „Zero-shot“-Generalisation → SOTA bei 7/8 NLP-Tasks ohne spezifisches Fine-Tuning.

BERT (Devlin et al., 2019)

- **Bidirektionaler Encoder-Transformer**.
- Pretraining-Aufgaben:
 - **Masked Language Modeling (MLM):** Vorhersage maskierter Tokens.
 - **Next Sentence Prediction (NSP):** Erkennen von Satzbeziehungen.
- Ergebnis: SOTA auf 11 NLP-Tasks (z. B. GLUE Benchmark).



Erweiterungen

- **Transformer-XL:** bessere Modellierung von langen Kontexten durch rekurrente Speichersegmente.
- **XLNet:** Autoregressive Pretraining-Variante, die Permutationen von Wortreihenfolgen nutzt.
- **RoBERTa:** optimierte Trainingsstrategie (mehr Daten, längeres Training).
- **T5, BART:** Encoder-Decoder-Transformers für Text-to-Text-Aufgaben.